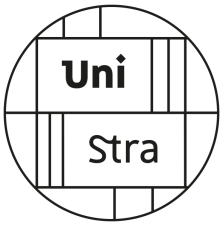

Quantum Bluff

Final Project Report

University of Strasbourg

Bachelor of Science in Computer Science (L3)

Supervised project — reference specification: `Docs/Cahier_de_charge_6A_PI.pdf`

Academic supervisor / Encadrant pédagogique: Jason Golda

Project manager: Omar
Team members: Linda, Massi, Abdoulaye, Soheil, Mohamed, Azra, Yigit

Academic year 2025–2026

Document generated: 14 May 2026

Abstract

Quantum Bluff is a capstone web platform produced for the third-year Bachelor's degree in Computer Science (L3) at the University of Strasbourg. It combines an authoritative Texas Hold'em stack with extended casino-style modules (solo and multiplayer blackjack, slots, European roulette), structured tournaments with ancillary winner markets, social features (friends, invitations, peer loans), and progression systems (XP, daily challenges, leaderboards). This report explains how delivery evolved from an initial V-model framing to an agile-inspired workflow; which engineering tools preserved modularity and testability; and how runtime architecture separates the React client, Express REST endpoints, a Socket.IO gateway, Prisma-backed persistence, and Redis-backed coordination. Security controls (JWT and optional TOTP, segmented rate limits, idempotency on sensitive mutations, anti-cheat hooks) and an automated test harness are summarized alongside appendices that map each major domain to repository artefacts. The product uses **virtual chips** for pedagogical demonstration; it does not implement real-money gambling.

Résumé

Quantum Bluff est une plateforme web temps réel développée dans le cadre du projet de fin d'études de L3 informatique à l'Université de Strasbourg. Elle couvre le poker Texas Hold'em (tables cash, bots, paris latents), des modules type casino (blackjack solo et multijoueur, machine à sous, roulette européenne), des tournois structurés avec marchés annexes sur le vainqueur, des fonctions sociales (amis, invitations, prêts entre joueurs) et de la progression (XP, défis quotidiens, classements). Ce rapport présente l'évolution de la gestion de projet (phase en V initiale puis itérations inspirées du Scrum), les choix technologiques, l'architecture logicielle, la sécurité et les résultats des tests automatisés. Des annexes indexent les chemins du dépôt pour chaque sous-système. Le système repose sur une **monnaie virtuelle** à visée pédagogique et ne constitue pas un service de jeu d'argent réel.

Keywords: multiplayer games, Texas Hold'em, real-time web, WebSocket, TypeScript, React, Node.js, Express, Prisma, PostgreSQL, Redis, Socket.IO, software engineering, security, automated testing.

Mots-clés : jeux multijoueurs, Texas Hold'em, web temps réel, WebSocket, TypeScript, React, Node.js, Express, Prisma, PostgreSQL, Redis, Socket.IO, génie logiciel, sécurité, tests automatisés.

List of abbreviations

API	Application Programming Interface (HTTP/JSON in this project).
CI/CD	Continuous integration and continuous deployment (GitLab CI at repository root).
HUD	Heads-up display: in-table probability and hand-category feedback.
JWT	JSON Web Token used for HTTP and Socket.IO authentication.
ORM	Object–relational mapping (Prisma).
REST	Representational state transfer style for `/api/*` routes.
RTK	Redux Toolkit and RTK Query for client-side data fetching.
TOTP	Time-based one-time password (optional second factor).
TTL	Time-to-live for cached or in-memory sessions.
UI/UX	User interface and user experience layers in the React SPA.
2FA	Two-factor authentication (TOTP in production paths).

Table of contents

List of Figures

List of Tables

1 Introduction

1.1 Context and Objectives

Quantum Bluff was developed as part of the final third-year Bachelor's degree project (L3 Computer Science, French academic system) at the University of Strasbourg. The objective was to deliver a **real-time multiplayer gaming platform** with a professional-grade engineering posture: modular architecture, automated testing, observability hooks, security hardening, and collaborative delivery under time constraints.

The **initial product intent** (before scope expansion) was a Texas Hold'em poker platform: practice mode against AI with multiple difficulty levels, multiplayer cash tables (public/private), configurable blinds, minimum buy-in rules, a probability-oriented HUD, and a **hidden side-bet** subsystem. The technical goals included authoritative server-side rules, transactional chip movements, WebSocket-scale event delivery, and operational readiness (health checks, metrics, recovery paths). The related technical appendices are Appendix D, Appendix E, Appendix F, Appendix G, and Appendix I.

Security and reliability were first-class constraints from the outset: JWT authentication, optional TOTP 2FA, anti-cheat and rate-limiting layers, idempotent handling for sensitive mutations, and schema validation on critical API paths. Implementation details are grouped in Appendix O, Appendix I, and Appendix P.

1.2 Scope of this document

This report summarizes **management evolution**, **tooling**, **delivered capabilities**, and **architecture**. Exhaustive technical narratives (bots, probabilities, tournaments, hidden bets, real-time synchronization, security details, etc.) are deferred to the **appendices** to avoid repetition. Repository-relative file paths use `\path{...}` notation. The front matter includes an abstract (English and French), a list of abbreviations, the table of contents, and lists of figures and tables.

2 Methodology, traceability, and reading guide

2.1 Traceability to the initial specification

The baseline expectations for the Bachelor project are captured in the repository specification `Docs/Cahier_de_charge_6A_PI.pdf`. The delivered system **extends** that poker-centric charter: all original poker-facing commitments (cash tables, bots, HUD, hidden bets, real-time play) remain grounded in server authority, while additional modules (casino mini-games, tournaments, social lending, progression) were integrated under the same engineering constraints (tests, rate limits, transactional chip movements). Where the PDF and the code diverge, **the implementation and this report defer to the repository as the operational source of truth**, with internal notes under `Docs/detail_logique/`, the repository map in Appendix A, and the relational database overview in Appendix C providing rationale.

2.2 Evidence hierarchy

Claims in the body are intentionally **outcome-oriented**. File paths anchor verifiable detail. Long-form auto-generated material (for example `Docs/Rapport_final/RAPPORT_TECHNIQUE_COMPLET.md`) is treated as an *index*: useful for breadth, but not cited as automatically validated against every line of code.

2.3 How validation is reported

Automated suites and manual smoke scenarios are summarized in Section 10 and cross-checked against `Docs/RAPPORT_TESTS.md` and `Docs/rapport-validation.md`. Playwright end-to-end tests live under `client/e2e/`. CI behaviour is defined in `.gitlab-ci.yml`.

3 Project Evolution

3.1 Initial specifications (poker-first)

The early scope targeted: Texas Hold'em cash games; **AI practice** with four difficulty tiers (`easy`, `medium`, `hard`, `expert`) enforced in `server/src/routes/game.api.routes.ts` and `server/src/routes/bot.routes.ts`; public/private multiplayer rooms with SB/BB configuration and minimum balance rules; **real-time** updates via Socket.IO; a **Quantum HUD** for win-equity and hand-category feedback (`client/src/components/QuantumHUD.tsx`); and **hidden bets** resolved server-side (`server/src/poker/hiddenBets/`). See Appendices D, E, F, G, and I.

3.2 Scope expansion (casino, tournaments, social, progression)

The delivered system adds: **Blackjack** solo and **multi-table** lobbies (`client/src/pages/Blackjack.tsx`, `client/src/pages/BlackjackMultiLobby.tsx`, `client/src/pages/BlackjackMultiTab tsx`); **slot** and **roulette** flows reachable from the mini-games hub (`client/src/pages/MiniGames.tsx`); **tournaments** with waiting room, in-tournament table, results, and **winner prediction side bets** (`client/src/features/tournament/`); **leaderboards**; **XP**, badges, daily challenges, daily login streak; **friends**, invitations, join requests, **peer loans** with interest/repayment hooks; **profile** editing with server-stored avatars; **2FA TOTP**; **feedback** and **player reports**; **wallet ledger**, gift codes, and dev recharge helpers; **i18n** in five locales (`client/src/i18n/locales/`); **tutorial** flow (`client/src/pages/TutorialGame.tsx`, `client/src/features/tutorial/`); **admin web console** and packaged targets (**Electron**, **Capacitor**) per `client/package.json`. The corresponding detail is split across Appendices J, K, H, L, M, O, N, and P.

3.3 Why the narrative moved from "catalog" to "appendices"

The codebase grew across **client**, **server**, **database**, and **docs/** technical notes. Listing every subsystem in prose would duplicate `docs/ARCHITECTURE.md` and the structured logic notes under `Docs/detail_logique/`. The body therefore stays outcome-oriented; Appendix A maps each major subsystem to code anchors, and Appendices D–S provide the deep technical material.

4 Project Management

4.1 Initial V-model phase

The project began with a **V-model style** framing: business requirement gathering with end users in mind, benchmarking comparable products, brainstorming sessions **validated collectively** by the team, and a written requirements package plus an initial Gantt chart. A **technical lead** was appointed to structure the MOE (implementation) track while the project manager covered the MOA (needs, prioritization, specification) track until the specification baseline stabilized.

4.2 Transition to an agile-inspired workflow (Jira, Scrum practices)

After an intensive early phase (approximately **two months** of V-model-oriented execution on a **four-month** calendar), the team concluded that strict phase separation slowed integration of new gameplay and infrastructure ideas. The organization shifted toward an **iterative, Scrum-inspired workflow**: **Jira** for epics, user stories, and time-boxed sprints; backlog ordering by **value vs. risk**; twice-weekly synchronization meetings for decisions, unblockers, and demo-ready increments. The project manager studied agile practice material (including *Scrum Life*) to align rituals with the team's reality rather than copying a textbook process wholesale.

4.3 Planning, coordination, and reporting

Coordination relied on: recurring meetings (~twice per week), shared technical documentation under `Docs/`, internal validation sessions, and continuous integration feedback via `.gitlab-ci.yml` at the repository root. Work was distributed dynamically across frontend, backend, and infrastructure tasks because multiplayer features require tight coupling between UI state, socket events, and persistence.

4.4 Product management mechanics

Backlog prioritization balanced user-visible value (new game modes, social features) against enabling work (Redis adapter, rate limits, recovery jobs, observability). Cross-cutting concerns (security, idempotency, anti-cheat) were escalated early to avoid late integration risk.

5 Development Tools and Technologies

5.1 Frontend and backend

Client side. The frontend is a React and TypeScript single-page application built with Vite. This choice gave the team a fast development loop, typed components, and a clean way to split the application into gameplay pages, lobby flows, profile screens, administration screens and reusable UI components. Tailwind CSS and Radix UI primitives were used to keep the interface consistent while still allowing custom casino-themed layouts. Redux Toolkit with RTK Query centralizes API calls, while the Socket.IO client handles live gameplay, invitations and presence updates. Route and provider mapping is summarized in Appendix A.

Server side. The backend is a Node.js and Express application with Socket.IO for real-time traffic. It exposes REST endpoints for durable commands such as authentication, profiles, wallet actions, tournaments and social features, and uses WebSockets for live table updates. Prisma gives typed access to PostgreSQL, which remains the durable source of truth. Redis is used where temporary coordination is needed: rate limits, token revocation, Socket.IO horizontal broadcasting and operational fallbacks. Security-related tools include Zod validation, JWT authentication, bcrypt password hashing and HTML sanitization where user content may be displayed. Socket and authentication details are expanded in Appendices I and O.

5.2 Database and infrastructure

PostgreSQL stores users, balances, games, tournament state, social relationships, reports, rewards and audit-friendly records such as wallet ledger entries. Redis is deliberately not used as the main database: it supports short-lived and distributed behavior, while PostgreSQL keeps the authoritative history. Docker Compose supports local development, and the VM deployment uses containerized services behind Nginx with HTTPS and WebSocket forwarding. Operational setup is detailed in Appendix P.

5.3 Version control and collaboration

The project was coordinated through Git, merge requests and GitLab CI. The CI pipeline builds, tests and packages the application so regressions are detected before deployment. Container images are built with Kaniko, which is useful on shared GitLab runners because it can build OCI images without requiring a privileged Docker daemon. The VM then pulls the validated images from the GitLab Container Registry. Documentation and logic notes live under `Docs/`; CI and packaging details are in Appendix P.

5.4 Testing and deployment

The server test suite uses Jest and Supertest for logic and integration behavior, while the client uses Vitest for focused frontend units and Playwright for browser-level smoke tests. Deployment is primarily web-based, but the React application can also be wrapped with Electron for desktop builds and Capacitor for mobile shell projects. This packaging work

does not change the core architecture: all versions still rely on the same backend authority, database and WebSocket synchronization. See Section 9 and Appendix P.

6 Use of Artificial Intelligence

6.1 Programmed in-game poker AI (bots)

The product ships **four deterministic difficulty tiers**—`easy`, `medium`, `hard`, `expert`—implemented entirely in TypeScript (`server/src/logic/botAI.ts`) and exposed through `server/src/routes/bot.routes.ts`. Each decision receives the visible hole cards, community cards, pot geometry (`potSize`, `callAmount`, `minRaise`), stacks, seat index, and player count. A shared primitive maps any partial board to a scalar strength by normalising the evaluator score against the theoretical maximum ($S_{\max}$ in Appendix E). Easy and medium tiers inject tunable randomness (loose calls, naive bluffs); hard tightens fold thresholds; **expert** adds deeper pot-odds heuristics before any optional remote call.

6.2 Expert tier: Python oracle and local oracle

`server/src/services/botAi.service.ts` wraps `decideBotAction`. For `expert`, the service first tries an **optional HTTP micro-service** when `AI_SERVICE_URL` is set: it POSTs a compact JSON payload (holes, board, stacks, street, etc.) to `/predict/poker`, validates the JSON response with **Zod** (`action`, `amount`, `confidence`, `style`, `reason`), converts `ALL_IN` into legal raise/call/fold semantics, and logs latency. If the URL is unset, the HTTP call fails, or validation fails, the server **falls back** to the pure-TypeScript `expertBotDecision`.

When hole cards of opponents are known (practice/oracle contexts), `expertOracleDecision` compares the hero strength to a **composite opponent strength** built from sorted opponent strengths $\{s_1 \geq \cdots \geq s_n\}$: for $n \geq 3$ it blends the best and median components with a player-count-dependent weight w (Appendix E), then applies pot-odds and pressure ratios to choose value raises, bluffs, calls, or folds. This path avoids the "always fold" failure mode of heuristics that ignore revealed cards.

6.3 Expert neural policy (PyTorch MLP, optional service)

When `AI_SERVICE_URL` points to the FastAPI bundle under `server/ai-service/`, the expert path can run a **trained multilayer perceptron** (`ExpertPolicy` in `server/ai-service/poker/train.py`, weights serialised to `server/ai-service/model/expert_bot.pt`). The vectoriser `features.py` exports `FEATURE_NAMES`: **36** numeric descriptors (streets, equity, pot odds, SPR, board texture, action frequencies, etc.). The network stacks **four linear layers** separated by ReLU: $36 \to 96 \to 64 \to 32 \to 4$ logits. Counting post-input activation units (hidden + output heads) gives $96 + 64 + 32 + 4 = 196$ **neurons**. Dropout (p=0.06 then 0.04) is applied after the first two hidden blocks during training only. The four outputs correspond to the internal action indices distilled from the expert teacher (`expert_rules.py`); inference-time mixing with heuristics and temperature sampling is described in `server/ai-service/poker/decision.py`. The compact trainable weight count in the linear stack is **11 972** scalar parameters (weights plus biases). Figures 1 and 2 show **flowcharts**: tensor pipeline inside the expert policy, then the Node↔Python decision path with branching. Appendix E contains the heuristic fallback and dispatch details.

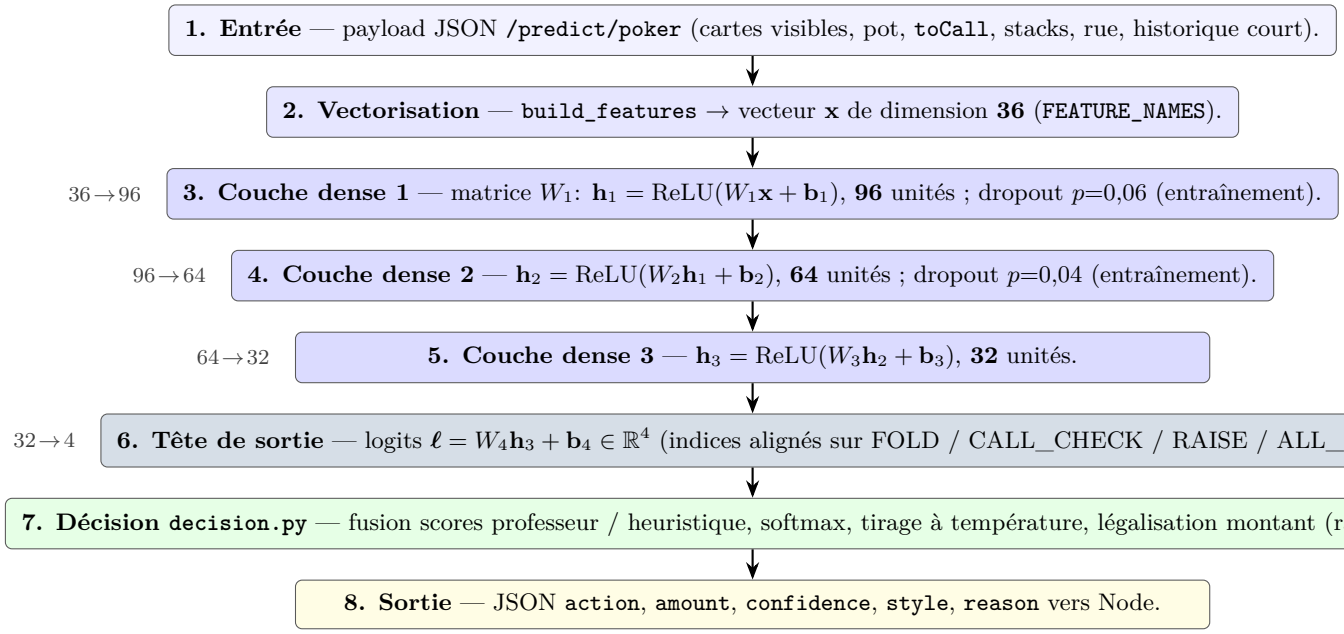

Figure 1: Flux de données à travers la politique MLP expert (`ExpertPolicy`): 4 transformations linéaires successives, **196** neurones après la couche d'entrée (96+64+32+4), puis post-traitement métier avant réponse HTTP.

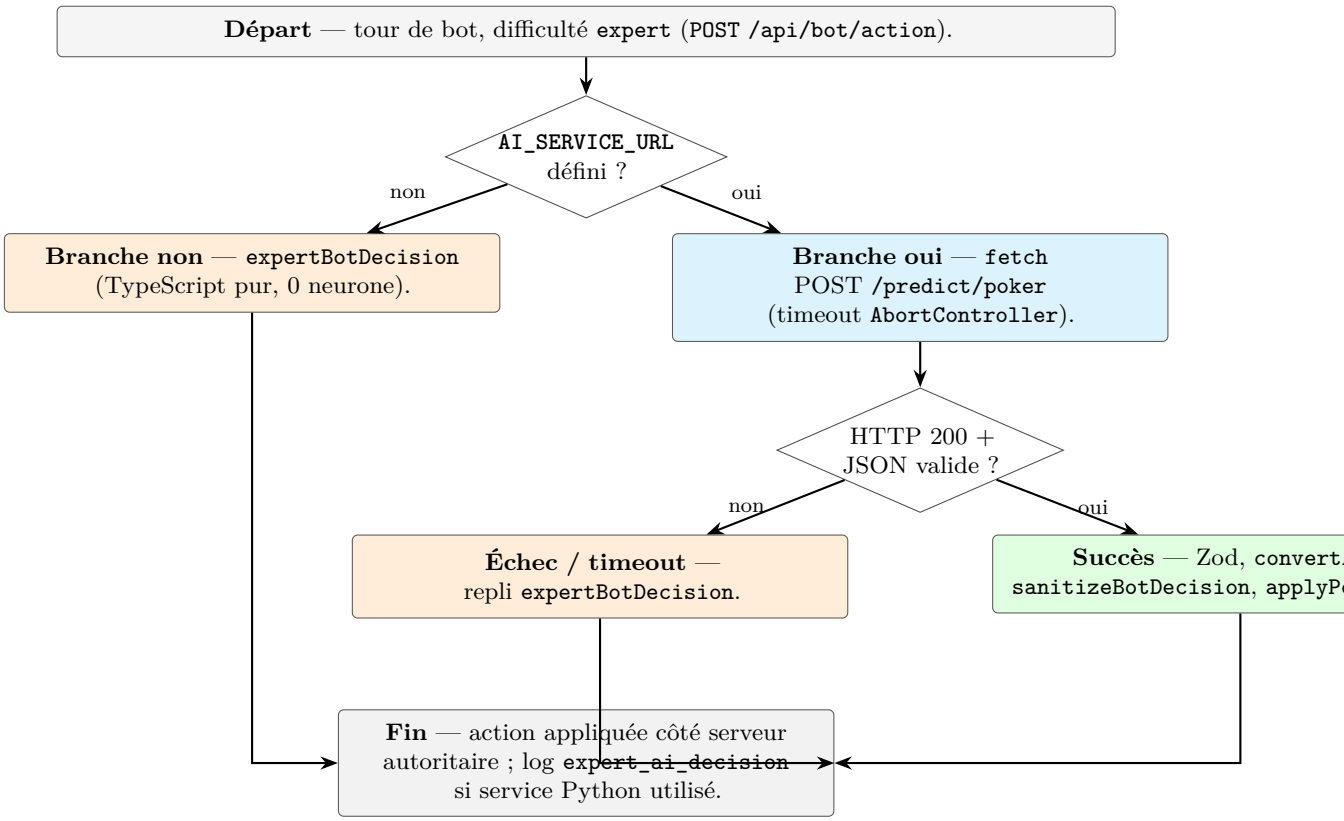

Figure 2: Diagramme de flux décisionnel côté Node pour le niveau `expert` (orchestration `botAi.service.ts`, repli heuristique, durcissement).

6.4 AI-assisted engineering (tooling)

Team members used modern **coding assistants** (e.g., IDE-integrated LLMs) for drafting, refactors, and documentation. Generated output was treated as **untrusted**: reviewed in merge requests, validated by automated tests, and rejected when it contradicted server authority or security invariants.

6.5 Limits and verification

The `easy`/`medium`/`hard` tiers and the TypeScript `expert` fallback contain **no trainable neurons**: they are hand-crafted policies. Only the optional Python bundle instantiates the MLP in Figure 1. Regardless of path, bot traffic is **rate-limited**, responses pass through **Zod and sanitisation** before touching authoritative state, and assistant-generated repository edits were never merged without human review and automated tests.

7 Platform Functionalities

Table 1 lists major domains. Details appear only once, in the referenced appendix, to prevent duplication. **Appendices A–S** also record implemented formulas and structures: relational database model, closed-form **slot RTP** and unified **roulette RTP**, Monte Carlo equity with an error bound for the **Quantum HUD**, **Quantum Combi** exact probability plus its place in the hand-ranking cascade, **tournament bracket** sizing and RNG, **code coverage** definitions, and the source files where each is coded.

Table 1: High-level functional inventory (representative surfaces).

Domain	User-visible capability	Primary UI entry	Deep dive
Poker cash	Multiplayer tables, pot logic, chat, actions	`/game`, `/waiting-room`	Appendix D
Poker bots	Practice tables, four difficulties	`/bot-configuration`	Appendix E
HUD	Win chance and hand category estimates	In-game HUD	Appendix F
Hidden bets	Side markets, pricing, settlement	In-game panel, `/results`	Appendix G
Blackjack	Solo + multi lobby/table	`/blackjack`, `/blackjack/lobby`, `/blackjack/table/:gameId`	Appendix J
Casino	Slot and roulette	`/minigames`	Appendix K
Tournaments	Waiting, play, results, winner side bets	`/tournaments/...`	Appendix H
Social	Friends, invites, loans	`/friends`, lobby flows	Appendix L
Progression	XP, badges, daily challenges, login streak, leaderboard	`/profile`, `/leaderboard`	Appendix M
Account	Auth, 2FA, avatars, profile edit	`/auth`, `/edit-profile`	Appendix O
Admin	Operator web console (JWT admin role)	`/admin/console`	Appendix P
Ops	Metrics, health, CI, packaging	N/A (infra)	Appendix P

Domain	User-visible capability	Primary UI entry	Deep dive
Other UX	Gift codes, free recharge, post-game rating modal	`client/src/components/Layout.tsx`, lobby	Appendix P
Feedback	User feedback and player reports	Lobby/settings flows	Appendix P
Tournament UX	ZipRush mini-game while waiting	`client/src/features/tournament/components/ZipRushMiniGame.tsx`	Appendix H
Demo	Example and deal teaching routes	`/game-example`, `/game-deal`	Appendix D
A11y	Accessibility, table theming, ambient audio	`client/src/contexts/AccessibilityContext.tsx`, `TableThemeContext.tsx`, `MusicContext.tsx`	Appendix A

7.1 Poker and Tournament System

7.1.1 Functional overview

Poker is implemented as an **authoritative server game**: the client sends intentions, while the server validates turns, blinds, raises, folds, pots, side pots and showdown results. The tournament layer reuses this poker engine and adds registration, entry fees, seeded bracket generation, table assignment, elimination tracking, final rewards and winner side bets. The full poker engine and tournament lifecycle are detailed in Appendices D and H.

- **Poker core**: `GameTable.ts` and `CashGameController.ts` enforce legal actions and chip movement.
- **Tournament orchestration**: `server/src/tournament/` creates rounds, tables and transitions.
- **Real time**: Socket.IO rooms notify players about table assignment, next rounds, eliminations and completion.

- **Reproducibility**: Mulberry32 + Fisher–Yates gives the same bracket for the same players and seed.

7.1.2 Four-phase tournament lifecycle

Phase	Activities
`REGISTRATION_OPEN`	Players join and entry fees (10% of stack) are deducted atomically. Fees accumulate into a centralized prize pool. Player state: `REGISTERED`.
`STARTING`	Bracket is generated deterministically. Player assignment is seeded and reproducible. Tables are dynamically sized (2–9 players each). Player state: `ACTIVE`.
`IN_PROGRESS`	Multiple poker games execute concurrently at separate tables. Losers are eliminated and ranked. Winners advance to the next round.
`COMPLETED`	Final rankings determine XP rewards and prize distribution. All mutations are atomic via Prisma transactions.

7.1.3 Multi-pot settlement: fairness under all-in

The same side-pot logic is used in cash and tournament tables. If three players are all-in for 100, 150 and 200 chips, the pot is split into a main pot and side pots so that each player can only win chips they were eligible to contest:

1. Extract unique bet levels in sorted order: $\{100, 150, 200\}$.
2. For each level ℓ, compute increment $\Delta_i = \ell_i - \ell_{i-1}$ and pool contribution $\Delta_i \times n_{\text{players}}$.
3. Distribute each pool only among players who contributed at that level.

This deterministic splitting keeps all-in situations fair and testable. Implementation details are in Appendix H.

7.1.4 Security and performance

Entry fees, rewards and winner-bet settlements run inside Prisma transactions. Action deduplication (`handId + actionId`) prevents duplicate mutations after packet retransmission. Game snapshots are persisted after actions and can be restored after restart. Table sizes adapt to player count (for example $[4, 4]$ for 8 players and $[5, 4]$ for 9 players). Appendix H gives the bracket arithmetic, RNG seeding and tournament flow diagram.

7.2 Casino Games and AI Bot Integration

7.2.1 Scope and functionality

Beyond poker, Quantum Bluff integrates three casino games—blackjack (solo and multiplayer), European roulette (37-pocket wheel), and a three-reel slot machine—plus a four-tier AI bot system (easy, medium, hard, expert) for practice and entertainment. Each casino game is engineered with closed-form return-to-player (RTP) calculations: slots deliver 95.63% RTP (house edge 4.37%), roulette delivers 97.297% RTP (house edge 2.703% per bet line), and blackjack approximates 99.3% RTP under basic strategy. All outcomes are computed server-side, persisted to an immutable wallet ledger, and broadcast to clients via WebSocket; clients never "decide" results. Mathematical and implementation details appear in Appendices J, K, E, and N.

The AI bot framework provides practice opponents across all four poker difficulty tiers. Bots participate in cash games (sit-and-go), tournament tables, and practice-only tables. They synthesize hand strength heuristics (normalized sigma), pot odds, stack pressure, and per-tier decision templates (bluff vs. value) to produce strategically coherent play. The expert tier optionally delegates to an external PyTorch model for advanced decision-making; on network failure, it gracefully reverts to the hard tier.

7.2.2 Architecture: authoritative server and real-time delivery

The casino subsystem follows the same authoritative server pattern as poker:

1. **Client submission**: player places a bet (blackjack, roulette, or slot spin) or bot decides action via WebSocket event (e.g. `PLACE_ROULETTE_BETS`, `SPIN_SLOT`).

2. **Server validation**: backend validates bet size, balance sufficiency, and game state consistency within a Prisma transaction.

3. **Outcome computation**: server uses injected RNG (`server/src/rng/rng.service.ts`) to draw the outcome (roulette pocket, slot symbols, blackjack card); for bots, it calls `decideBotAction` with normalized hand strength.

4. **Ledger recording**: a wallet ledger entry (`GAME_WIN` or `GAME_LOSS`) is created atomically with the balance update, capturing user ID, reason, amounts, and ISO timestamp.

5. **WebSocket broadcast**: outcome is emitted to the player's room (e.g. `ROULETTE_RESULT { pocket, payouts }`) so the client renders the result without delay.

6. **Snapshot persistence**: game state (dealer hand, final balances) is snapshotted to PostgreSQL for crash recovery.

This architecture guarantees consistency: outcome and ledger entry commit together, or neither commits. Clients cannot force outcomes; they can only place bets and observe server decisions. The same transaction pattern is expanded in Appendix N.

7.2.3 AI bot decision algorithm and complexity analysis

The core of the bot framework is the **normalized hand strength** (sigma) computation and multi-tier decision dispatch. Let the bot's visible cards (two to seven) yield a raw hand evaluation score H via `getHandValue`. The code normalizes to the unit interval:

$$\sigma = \min\Big(1,\, \max(0,\, \frac{H}{S_{\max}})\Big), \quad S_{\max} = 9 \cdot 15^5 + 14 \cdot 15^4,$$

where $S_{\max}$ is the theoretical best hand (straight flush with top kickers). This ensures $\sigma \in [0, 1]$ regardless of the board texture or number of opponents. Appendix E gives the tier-specific decision paths.

The bot then dispatches to a tier-specific decision function:

$$\texttt{decideBotAction}(\sigma, \texttt{potOdds}, \texttt{stackPressure}, \text{tier}) \to \{\texttt{FOLD}, \texttt{CALL}, \texttt{CHECK}, \texttt{RAISE}\}$$

Each tier has distinct complexity:

- **Easy**: compares σ to a fixed threshold; injects naive bluffs with probability $p_{\text{bluff}} = 0.15$ on free streets. Complexity: $\mathbf{O(1)}$.
- **Medium**: integrates pot odds callAmount/(potSize + callAmount); adjusts bluff frequency by position (button $p = 0.20$, under-the-gun $p = 0.08$). Complexity: $\mathbf{O(1)}$.
- **Hard**: maintains opponent hand range estimates (map of recent action patterns). Decision compares σ against estimated opponent strength distribution. Complexity: $\mathbf{O(n)}$ where $n \leq 8$ (number of opponents).
- **Expert**: HTTP POST to external model (`.../predict/poker`) with JSON payload (compact card encoding, street, stacks, position). Zod-validated response; timeout (default 2000 ms) reverts to hard tier. Complexity: $\mathbf{O(1)}$ on the bot side (async); external model latency is hidden.

All tiers converge on the same sigma input, enabling principled interpolation from naive to advanced strategy. The hard tier's $O(n)$ opponent tracking is acceptable because n is small (typical tables have 2–9 players).

7.2.4 Integration with poker and tournaments

Casino games are separate from the poker state machine but share the wallet ledger and user authentication. Bots can be inserted into poker cash games (via `server/src/routes/waitingRoom.routes.ts`) or tournament tables (via `server/src/tournament/tournamentTableFactory.ts`) by specifying a difficulty tier at table creation. The same bot decision logic applies; the only difference is the hand context (hole cards, board, position) provided by the GameTable or BlackjackTableController runtime.

Full algorithmic details, RTP derivations, and per-tier heuristics are provided in Appendix E (Bot AI), Appendix J (blackjack), and Appendix K (slots and roulette).

7.3 Rewards, Banking, and Transactions

7.3.1 Scope and architecture

The rewards and banking system orchestrates chip circulation, XP progression, and peer-to-peer lending across all game modes: poker cash games, casino games, tournaments, and social loans. All transactions are recorded immutably in a **wallet ledger** (`server/src/casino/services/walletLedger.service.ts`) with cryptographic integrity hashing (SHA-256), ensuring forensic auditability and regulatory compliance. The system enforces **server-side authority**—clients never compute balance changes; they request actions and receive confirmed outcomes via WebSocket. Every chip movement is atomic: ledger entry and balance update commit together via Prisma transactions, or neither commits. See Appendix N for ledger fields and settlement examples.

7.3.2 Gamification: XP curves, levels, and badge unlocks

Player progression is governed by accumulated experience points (XP) and a deterministic level curve. The core formula is:

$$\text{XP threshold for level } L = 50 \cdot L \cdot (L-1), \quad L \in \{1, 2, \ldots, 99\}$$

XP is awarded per action: poker (12 XP/hand + 20 bonus on win), blackjack (5 XP + 10 bonus), roulette/slots (4 XP + 8–10 bonus), tournaments (500 / 400 / 300 / 50 XP for placement). Badge unlocks are level-driven (10 tiers: novice→legend). Effective betting limits scale by level to prevent new players from wagering excessively: slot max bet grows from 500 to 5000 chips as level increases (capped at 25). Appendix M records the progression modules.

7.3.3 Wallet ledger and atomic transactions

Every chip movement—game win/loss, loan disbursement, loan repayment, tournament prize—creates an immutable `WalletLedgerEntry` row in PostgreSQL. Fields include: userId, reason (e.g. `GAME_WIN`, `TOURNAMENT_PRIZE`, `LOAN_FUNDED_OUT`), amount, balanceBefore / balanceAfter, roundId (for traceability), integrityHash (SHA-256 of all fields + versions), and settlementState (`SETTLED` or `PENDING`). For a casino game, the sequence is: (1) validate bet size and balance, (2) compute RNG outcome, (3) within `prisma.$transaction`: update `User.chips`, create ledger entry, compute integrity hash. This guarantees that ledger and balance are always synchronized; Appendix N lists the persisted fields.

7.3.4 Tournament rewards and peer loans

Tournament prize pools sum all entry fees (typically 10% of initial stack). Winners receive the pool; all participants receive placement-based XP. Reward grants are idempotent via a `TournamentRewardLedger` unique constraint, enabling crash recovery. Peer-to-peer loans between friends specify a principal P and repayment rate $r\% \in \{10, 15, \ldots, 50\}$. The system converts r to an annual interest via a lookup table (e.g. 10% repayment → 30%

interest). When the borrower wins chips, the server automatically diverts a slice to the lender: slice = min(remaining due, $\lfloor \text{grossWin} \cdot (r/100) \rfloor$), capped at the amount owed. Repayment is mandatory; the loan settles atomically. Constraints: borrower and lender must be friends, borrower can have at most one active loan, lender must have sufficient chips at acceptance, rate-limited to 40 operations per 10 minutes. Tournament reward details are in Appendix H; loan and ledger details are in Appendices L and N.

7.3.5 Integration with daily challenges and login streaks

Daily challenges and login streaks (separate modules in `server/src/dailyChallenges/`, `server/src/dailyLogin/`) award XP using the same `awardXp` function, feeding into the gamification bundle. Client displays these in `/profile` and `/leaderboard` routes. Appendix M links the relevant modules.

8 System Architecture

8.1 Monorepo layout

The repository splits into `client/` (Vite SPA), `server/` (Express + Socket.IO gateway), and `database/` (local Postgres). Client bootstraps providers in `client/src/main.tsx` and routes in `client/src/App.tsx`. Server composes middlewares and REST mounts in `server/src/index.ts`, then attaches `server/src/sockets/game.gateway.ts` to the HTTP server. Appendix A provides the repository map.

8.2 Central architectural model

Quantum Bluff is organized around a **layered authoritative architecture**. The React client is responsible for navigation, rendering, forms, local interaction state and immediate feedback. It never owns final gameplay decisions: poker actions, casino outcomes, tournament progression, wallet movements, hidden-bet settlement and rewards are all validated and applied on the server.

The backend is split into two entry surfaces. REST endpoints handle durable commands such as authentication, profile updates, wallet-related operations, tournament creation, social actions and history queries. Socket.IO handles live interactions such as joining tables, submitting turn actions, broadcasting sanitized snapshots, notifying tournament assignments and updating social presence. Both surfaces share the same identity model: JWT verification, role separation, suspension checks, rate limits and domain-level validation.

Behind these entry points, domain services isolate the main responsibilities: poker and cash-game rules, blackjack tables, casino mini-games, tournament orchestration, hidden bets, social features, rewards and wallet accounting. Persistent state is stored in PostgreSQL through Prisma, while Redis is used for short-lived coordination: token blacklist, rate limiting, Socket.IO horizontal broadcasting and operational fallbacks. This separation keeps the system readable: UI code displays state, gateway code routes events, domain services enforce rules, and the database remains the source of truth for identity, balances, history and audit trails.

Appendix B provides the full architecture diagram; Appendix C shows the relational data model behind the persistence layer.

8.3 Structured tournaments (outline)

Multi-table Texas Hold'em tournaments are **first-class server domain**: Prisma-backed registration and fees (`server/src/tournament/tournament.service.ts`), a periodic **scheduler** that auto-starts due events (`server/src/tournament/tournament.scheduler.ts`), runtime orchestration that builds bracket rounds and assigns `GameTable` instances (`server/src/tournament/tournament.runtime.service.ts`), deterministic **table splitting** and reseating (`server/src/tournament/bracket/`), winner-bet settlement (`server/src/tournament/winnerBets/`), and boot-time **recovery** (`server/src/tournament/tournament.recovery.service.ts`). The client module `client/src/features/tournament/` binds waiting room,

in-tournament play, results, and ZipRush. Exact bracket arithmetic, RNG seeding, and coverage metrics are in Appendices H and Q.

8.4 Real-time communication

Quantum Bluff uses Socket.IO for live gameplay, lobby updates, invitations, tournament events and social notifications. The browser sends intentions, but the server remains authoritative: it validates the authenticated user, the game state, the turn order, the chip balance and the action legality before broadcasting any result.

Synchronization is based on rooms and snapshots. Each user has a private `user:{userId}` room, while poker tables, blackjack tables and tournaments use dedicated rooms for shared updates. On join or reconnect, the server sends a complete sanitized snapshot instead of relying only on incremental events. This makes reconnection easier for the player and avoids rebuilding a game from stale partial events.

Confidentiality is also handled on the server. A seated player receives their own private cards, while other players and spectators receive a sanitized view. Redis support allows Socket.IO events to work across several server instances. The full event model, table locking, reconnection behavior and security checks are detailed in Appendix I.

8.5 Persistence and integrity

Prisma models capture users, games, tournaments, loans, hidden bet tickets, wallet ledger rows, and more (see `server/prisma/schema.prisma`). Critical monetary flows favor server-side orchestration and short transactions. Appendix N explains the ledger side of these models.

8.6 Data model overview (Prisma)

The persistence layer is relational PostgreSQL accessed through Prisma. Representative entities include: `User` (credentials, profile, chips, XP fields, avatar storage keys), `Game`/seat models for poker tables, `BlackjackRoom` and related seat/invitation models for multiplayer blackjack, tournament tables for bracket state and winner-bet tickets, `Friendship` and messaging hooks, loan and ledger tables for social/casino flows, and audit-friendly structures for wallet history. Full cardinality and migration history are only authoritative in `server/prisma/schema.prisma` and the migration folder; a simplified relational model is provided in Appendix C, and domain usage is split across Appendices D, J, H, L, M, and N.

8.7 Diagram

Figure 3 shows a **runtime flow diagram** (logical path, not deployment topology). For more detail, see `docs/ARCHITECTURE.md`, Appendix A, Appendix B, and Appendix I.

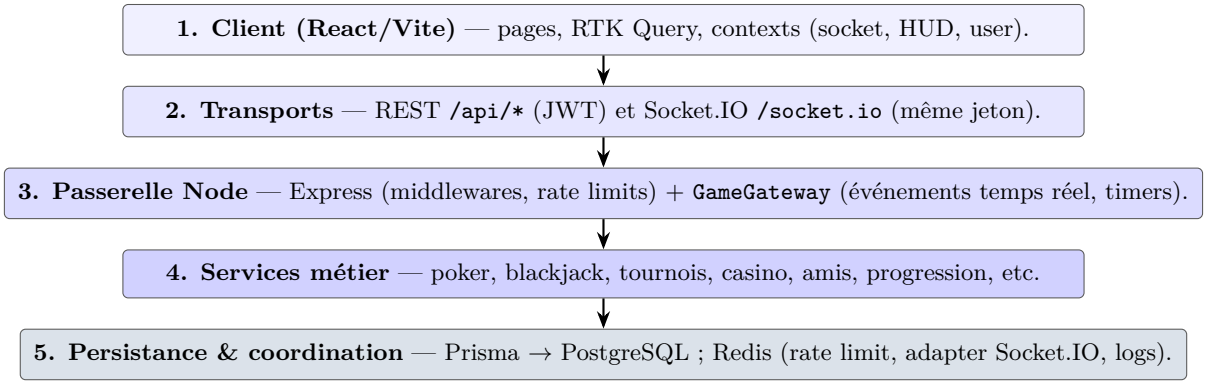

Figure 3: Flux d'exécution monorepo : interface → API / WebSocket → passerelle et services → données.

9 Deployment and operational runbook

9.1 Local development stack

The root script `npm run dev` (see `package.json`) launches PostgreSQL through `database/docker-compose.yml`, a lightweight helper script (`scripts/dev-ai.js`), the Node/Express server (`server/`), and the Vite client (`client/`) under `concurrently`. Redis should be available for rate limiting, action logging, and the Socket.IO Redis adapter when exercising multi-instance paths (`server/src/config/socketIoRedis.ts`). First-time setup, tester onboarding, and troubleshooting are documented in `README.md`, `Docs/SETUP_TESTEUR.md`, and `Docs/QUICK_TEST_GUIDE.md`; operational references are consolidated in Appendix P.

9.2 Production and desktop delivery

VM-oriented deployment (Node, PostgreSQL, Redis, PM2), environment files, and Electron auto-update mechanics are described in `Docs/DEPLOY.md`. The pipeline builds runnable **container images** on `develop`, `main`, and `release/*` using **Kaniko**: each job runs `/kaniko/executor` with `-context` pointing to `server/`, `client/`, or (on `release/*` only) the Python AI bundle under `server/ai-service/`, authenticates to the GitLab registry via `/kaniko/.docker/config.json`, and pushes immutable SHA-tagged images plus rolling `:latest`, `:client-latest`, or `:ai-latest` aliases so the VM can `docker pull` without rebuilding from source on the host. Reverse-proxy samples appear under `nginx/` when applicable. Appendix P links the deployment artefacts.

9.3 CI/CD pipeline and autonomous VM updates (GitLab Runner Shell mode)

The Quantum Bluff project uses a **multi-stage GitLab CI pipeline** (`.gitlab-ci.yml`) configured for **five stages**: verify (linting and unit tests), test-security (npm audit and Trivy scanning), build (transpilation), docker-build (Kaniko image construction), and deploy (staging and production). Appendix P records the CI/ops entry points, while Appendix Q defines coverage metrics.

9.3.1 Five pipeline stages

1. Verify: On `develop` and `main` branches, **backend:verify** runs Jest tests with coverage in a PostgreSQL 15 service; **frontend:verify** runs Vitest on the client; **database:test** smoke-tests the database. On `feature/*` and `fix/*` branches, **backend:lint** performs lightweight linting only.

2. Test-Security: **security:npm** audits lockfiles for high/critical vulnerabilities; **security:trivy** scans the filesystem with Trivy (skipping `node_modules`, `dist`, platform folders). Both allow failure to avoid blocking deployments.

3. Build: On `main` and `release/*`, **backend:build** generates Prisma client and transpiles to `server/dist/`; **frontend:build** bundles the Vite/React app to `client/dist/`.

4. Docker-Build: Kaniko builds three images: **docker:build** (backend tagged `:latest`), **docker:build:client** (frontend tagged `:client-latest` with `VITE_API_URL` build argument), and **docker:build:ai-service** (Python service, release/* only). All images are pushed to the GitLab Container Registry.

5. Deploy: **deploy:staging** automatically runs on `develop`; **deploy:production** requires manual approval on `main`.

9.3.2 GitLab Runner in Shell mode: the autonomous agent

The critical breakthrough that makes VM updates **fully autonomous** is a **GitLab Runner** installed directly on the deployment VM, configured in **Shell executor mode**. This solves the firewall constraint: the university's strict inbound policy blocks external connections to the VM. The Runner uses a **pull-based polling mechanism**:

1. **Polling:** Every 3 seconds, the Runner queries GitLab: "Are there new commits on `develop` or `main`?" instead of waiting for GitLab to push notifications inbound.
2. **Tag-based routing:** The deploy jobs in `.gitlab-ci.yml` use the tag `runner-vm-quantum`, telling GitLab: "Only the Runner with this tag can execute these jobs." This ensures the job stays in the queue until the VM's Runner claims it.
3. **Shell executor:** The Runner is configured with executor type `shell`, executing scripts directly on the VM. Docker socket access is granted via `sudo usermod -aG docker gitlab-runner`.

9.3.3 Automated deployment sequence

When the Runner claims a deploy job, it executes the shell script sequentially:

1. **Docker authentication:** `echo "$CI_REGISTRY_PASSWORD" | docker login -u "$CI_REGISTRY_US -password-stdin "$CI_REGISTRY"` — uses GitLab CI secrets to authenticate to the private registry.
2. **Dynamic .env generation:** Reconstructs `.env` from secure GitLab variables (`$JWT_SECRET`, `$DATABASE_URL`), ensuring secrets never exist in source code.
3. **Hot pull and restart:** `docker compose pull` fetches latest images, then `docker compose up -d -force-recreate` recreates containers with zero downtime.
4. **Database migrations:** `npx prisma migrate deploy` applies schema changes automatically inside the backend container.

9.3.4 Zero-downtime updates and autonomous operation

The `-force-recreate` flag on `docker compose up` tears down and recreates containers without affecting load balancing or WebSocket connections. The result is **fully autonomous CI/CD**: every `git push` to `develop` automatically stages code within seconds; every merge to `main` with manual approval deploys to production. The Runner polls for work, retrieves encrypted secrets from GitLab, and applies updates entirely within the VM's local context—no manual SSH, no cron jobs, no webhook proxies. Operational fallbacks and Redis-related constraints are referenced in Appendix P.

9.3.5 Further reading

`.gitlab-ci.yml`, `Docs/DEPLOY.md`, `server/Dockerfile`, `client/Dockerfile`, `docker-compose.prod.yml`; see also Appendix P.

9.4 Electron and Capacitor artefacts

The same React UI is built under three Vite modes: default web, `-mode electron`, and `-mode capacitor`. **Electron** (`client/electron.cjs`, `electron-builder` in `client/package.json`) targets **Windows** (NSIS → `.exe` installer), **macOS** (`.dmg` / signed artefacts per builder defaults), and **Linux** (`AppImage` and `.deb` as configured under `build.linux.target`). Published updates are served from the generic URL declared in `build.publish` for auto-update (`electron-updater`). **Capacitor** wraps the static build for **iOS** and **Android**: `npm run cap:sync` copies web assets into native projects; `cap:ios` / `cap:android` open Xcode and Android Studio. Repository scripts `scripts/package-ios-rendu.sh` and `scripts/package-android-rendu.sh` support course delivery packaging. Appendix P lists the npm entry points.

10 Security, Testing and Stability

10.1 Authentication and anti-cheat

Security is implemented as defence in depth. Short-lived JWT access tokens identify users on REST and Socket.IO paths; protected routes reject malformed, revoked, admin-role, missing-user or suspended-account tokens. Passwords are hashed with bcrypt, validated with Zod rules, and optional TOTP 2FA is available for stronger account protection.

Anti-cheat controls are applied at several levels. HTTP middleware checks bans and suspicious account patterns; the real-time gateway validates that a socket cannot act as another player; gameplay helpers detect abnormal action bursts; and moderation features persist reports and blocked-user relationships. Rate limits, CORS/Helmet configuration, idempotency for sensitive mutations and separated admin access complete the security baseline.

The result is a coherent academic prototype security model: identity is checked repeatedly, gameplay remains server-side, and suspicious behavior is logged or refused. The detailed token flow, socket identity checks, rate limits and anti-cheat mechanisms are provided in Appendices O, I, and P.

10.2 Testing and debugging

Broad automated coverage on server and client; Playwright smoke tests under `client/e2e/`. Additional manual scenarios are documented under `Docs/POKER_SCENARIOS.md` and related files. Appendix Q explains how test and coverage numbers are interpreted.

10.3 Automated test inventory and CI (reference snapshot)

Table 2 reflects a **May 2026** run (`cd server && npm test`, `cd client && npx vitest run`). Historical line counts also appear in `Docs/RAPPORT_TESTS.md`; refresh that file after large suite changes. Coverage definitions are in Appendix Q.

Layer	Runner	Test files / suites	Tests passed
Backend (`server/`)	Jest	51	520
Frontend (`client/`)	Vitest	13	83
Total unit/integration	—	64	**603**

Table 2: Automated unit and integration test inventory (May 2026 snapshot).

10.4 Code coverage policy (server)

Jest collects coverage only on **logic-heavy paths** listed in `server/jest.config.cjs` (`collectCoverageFrom` poker/tournament bracket utilities, validation, selected `src/utils`, etc.; some monolithic controllers are excluded so the metric tracks maintainable units). **Global thresholds** require at least 80% lines, statements, and functions, and 65% branches, or CI fails. Table 3 shows the latest measured aggregate on that scope. Appendix Q defines the coverage ratios and relates them to tournament bracket tests.

Layer	Lines	Statements	Functions	Branches
Server (Jest scoped paths)	86.56%	83.69%	88.75%	67.90%
Required minimum (threshold)	80.00%	80.00%	80.00%	65.00%

Table 3: Server Jest coverage aggregate on `collectCoverageFrom` (May 2026; `npm run test:coverage` in `server/`).

Client: running `vitest run -coverage` over the whole `src/` tree yields a **low global percentage** (~4% lines in a May 2026 run) because most UI and socket flows are covered by Playwright, manual poker scenarios, and integration with the live API rather than by isolated Vitest files. Per-file reports under `client/coverage/` still highlight modules with strong unit tests (e.g. `iban`, `avatars`).
GitLab CI configuration at `.gitlab-ci.yml` encodes build and test gates for merge requests; the `docker-build` stage uses **Kaniko** for reproducible, rootless image builds pushed to the GitLab Container Registry. A consolidated validation narrative for bot flows and UI smoke checks appears in `Docs/rapport-validation.md`. CI operational detail is in Appendix P.

10.5 Performance and stability

Redis-backed Socket.IO adapter, structured logging, Prometheus metrics endpoint, readiness endpoints, graceful shutdown hooks (`server/src/index.ts`). See Appendices I and P.

11 Technical Challenges

Concurrency around shared tables: poker and blackjack runtimes use locking strategies (see `server/src/poker/services/pokerTableLock.service.ts` and blackjack lock services) to prevent split-brain chip mutations. Related runtime details appear in Appendices D, J, and I.

Recovery after process restarts: blackjack and tournament subsystems implement recovery services invoked at boot (`server/src/index.ts`); see Appendices J, H, and I.

Hidden bet settlement coupling: market resolution must align with poker street transitions; implemented in hidden-bet resolver modules (Appendix G).

Internationalization without clobbering user preference: guest screens (StartScreen, Auth, global loader) force English using `react-i18next` with a fixed `lng: 'en'` override so the global language stored for authenticated use is unchanged (`client/src/pages/Auth.tsx`, `client/src/contexts/LoaderContext.tsx`); see Appendix O.

11.1 Implemented solutions

The final implementation is a modular real-time gaming platform rather than a single poker page. React handles the user interface, Express exposes REST APIs, Socket.IO delivers live events, Prisma/PostgreSQL stores durable state, and Redis supports coordination, rate limits, token revocation and distributed sockets.

The most important solution is server authority. Poker, blackjack, roulette, slots, tournaments, hidden bets, rewards and social money flows are validated and settled on the server; the client only displays state and sends user intentions. Chip-changing actions use persistence and ledger-oriented patterns so the database remains the source of truth. This same principle is applied to private poker cards, blocked-user warnings, invitations, reports, XP, badges, daily challenges and leaderboard updates.

The platform was extended with social and retention modules: friends, invitations, blocking/reporting, lobby warnings, peer loans, tournaments, winner side bets, XP, badges, daily challenges, leaderboards, gift codes and recharge flows. The lobby became the central operational screen after login, while gameplay pages remain specialized for poker, blackjack, roulette and slot machine flows.

Validation and tests were added around the most sensitive solutions: authentication schemas, trusted waiting-room identity, socket authentication, runtime snapshots, game logic and tournament utilities. The complete module-to-file mapping is intentionally kept in the appendices so the body remains readable: Appendices A, D, I, L, M, N, and Q.

12 Ethics, privacy, and academic integrity

12.1 Virtual economy and regulatory framing

Quantum Bluff uses **in-game virtual chips** for learning and demonstration. It is not a licensed real-money gambling operator; features that resemble casino products are implemented in an **academic engineering** context and should be evaluated as software artefacts, not as a commercial gaming offer. The casino payout models are documented in Appendix K.

12.2 Personal data

Accounts store identifiers needed for authentication and gameplay (email, username, hashed password, optional TOTP secrets, profile metadata). Operators deploying a public instance must align hosting, retention, and breach notification with applicable law (including GDPR for EU users). This report does not replace a data protection impact assessment. Appendix O lists the account/profile surfaces.

12.3 Integrity of authorship and AI assistance

Section 6 states how large language models were used as drafting assistants. All merged code remained subject to human review, tests, and repository governance. Each author is expected to sign off on their contribution statement in Section 13.

13 Individual Contributions

13.1 Omar – project management and architecture alignment

Organizational narrative: Section 4. Beyond ceremonies and backlog arbitration, I coordinated **cross-cutting technical commitments**: early adoption of Redis for Socket.IO horizontal scale-out and rate limiting, alignment between Prisma schema changes and gameplay features, and risk reviews before widening scope (tournaments, casino mini-games, loans). I also worked with the appointed technical lead to sequence risky merges, such as the socket gateway and recovery jobs, away from demo deadlines.

13.2 Massi – DevOps Infrastructure, Real-Time Networking, and Cross-Platform Delivery

I contributed to Quantum Bluff by working on DevOps infrastructure, real-time networking, deployment automation and cross-platform delivery. On the networking side, I helped design and stabilize the WebSocket/Socket.IO communication layer used by the poker tables and live gameplay flows. This included reconnection behavior, timeout handling, and client-side socket integration so that short network interruptions would not immediately break the player experience.

I also worked on the production infrastructure around reverse proxying, containerization and deployment. This included Nginx routing concerns for HTTP and WebSocket traffic, Docker-based service isolation, PostgreSQL and Redis deployment boundaries, and CI/CD automation through GitLab Runner. The goal was to make the platform deployable in a controlled environment while keeping database migrations, environment generation and service restarts reproducible.

Finally, I contributed to cross-platform packaging and release preparation. I worked on the Electron desktop target, Windows installer generation, and the structural setup needed to package the React application outside the browser. I also participated in bug hunting and stabilization work, especially around cross-origin issues, chat input handling and real-time state synchronization.

13.3 Yigit – Debugging, Quality Assurance and Gameplay Stability

I mainly contributed to debugging, testing and quality assurance throughout the development of Quantum Bluff. My work focused on identifying gameplay issues, reproducing bugs precisely, and validating that new implementations did not introduce regressions in the main user flows. I actively tested multiplayer poker games, tournaments and mobile responsive interfaces to ensure that the platform remained stable in realistic gameplay situations.

One of my main contributions concerned tournament synchronization and hand transition debugging. Several issues appeared during development, especially after showdown phases, automatic hand restarts and final table transitions. I tested edge cases, reproduced synchronization problems, and validated the implemented fixes until tournaments could continue

correctly without requiring page refreshes. I also checked that action buttons, player states and table transitions remained functional after each hand.

I also contributed significantly to responsive design improvements on mobile devices. During testing, I identified interface problems such as overlapping buttons, unreadable token displays and spacing issues between the poker table and gameplay controls. I helped optimize the layout for smaller screens while preserving the overall user experience and avoiding regressions on the desktop version.

In addition, I participated in Git integration, merge request validation and final testing sessions before deployment. This contribution helped improve the overall reliability, usability and quality of the final version of Quantum Bluff, especially on features that required real-time synchronization and repeated user interaction.

13.4 Azra – Backend Modeling, Visual Design and User Experience Features

I contributed to Quantum Bluff on both technical and visual aspects of the application. On the backend side, I helped structure the project by modeling important game entities in TypeScript and by participating in the implementation of card management logic and distribution rules inspired by classic casino mechanics. I also contributed to the tournament system by helping define its configuration, rules and expected gameplay behavior.

On the frontend side, I worked on the graphic design of several visible elements of the project, including playing cards and parts of the mini-game interfaces such as roulette and slot machines. These contributions helped make the application more recognizable and more consistent with the casino atmosphere of Quantum Bluff. I also contributed to accessibility improvements by working on colorblind mode support and by helping fix responsive design issues on multiplayer interfaces.

Finally, I contributed to several user experience and retention features, such as daily login rewards, the free virtual token recharge system and promotional codes. I also participated in testing, bug fixing and stability improvements before deployment. My work helped strengthen the visual identity of the project while improving the comfort, accessibility and completeness of the final user experience.

13.5 Linda – security, real-time features and UI/UX

I worked on backend integration, real-time features, security and UI/UX. My main technical contributions were game routes, blinds configuration, waiting-room fixes, player-state debugging, socket authentication, JWT hardening, stronger password rules, rate limits and anti-cheat checks.

I also worked on the friends and lobby experience: friend search, add-friend modal, friend cards, chat access, remove friend, block/unblock, user reports, invitations, live notifications, activity status and blocked-user warnings. On the UI side, I improved the lobby layout, reduced empty space and scrollbars, and contributed to profile/avatar, poker table, blackjack, roulette, mini-games, settings and responsive layout improvements.

13.6 Other contributors

Additional individual contribution statements for Abdoulaye, Soheil and Mohamed are managed separately by the team. This report does not assign detailed technical ownership to those members without their explicit validation, in order to keep the attribution section accurate and academically honest.

14 Limitations and Future Work

14.1 Current limitations

- **Documentation volume vs. examiners' time:** internal generated reports (e.g., `Docs/Rapport_final/RAPPORT_TECHNIQUE_COMPLET.md`) are useful as indexes but are not all human-curated line-by-line.
- **Dual loader providers:** `client/src/main.tsx` and `client/src/App.tsx` both wrap `LoaderProvider`; harmless but confusing—should be deduplicated.
- **Packaging surface area:** Electron/Capacitor increase release engineering work (signing, store policies) beyond the core web path; see Appendix P.
- **External identity and banking integrations:** the project does not include Google account verification, real email/SMS one-time-code delivery, real payment cards, or real bank account connections. These features require certified third-party providers, compliance checks, payment/banking approvals, and stronger legal guarantees than were available in the academic project context. The implementation therefore uses simulated top-ups, virtual chips, and internal authentication flows rather than real financial instruments.

14.2 Future improvements

Hardening continuous deployment secrets, expanding observability dashboards, deeper bot modeling (still code-first), richer tournament analytics, and replacing simulated identity/payment flows with certified providers if the project ever moves beyond an academic prototype. The related starting points are Appendices P, E, H, and O.

15 Conclusion

Quantum Bluff exceeded its initial poker-only charter by integrating multiple casino modes, tournaments, social lending, progression systems, and operational hardening while keeping a monorepo with clear separation between UI, API, and authoritative game logic. The project demonstrates that a student team can approach industry-like constraints—real-time state, monetary integrity, security, and observability—when management explicitly trades early waterfall rigidity for controlled agile iteration. The appendix set acts as the technical index for these domains, especially Appendices D–P.

Lessons retained for similar efforts: (1) anchor scope changes to **recoverable increments** (feature flags, migrations, recovery jobs); (2) treat generated documentation as **hypotheses** until cross-read against code; (3) keep **security and idempotency** on the critical path rather than as end-of-project polish.

Acknowledgements

The team thanks **Jason Golda** for academic supervision of this L3 capstone, the University of Strasbourg Computer Science faculty for the project framework, the open-source maintainers of the libraries listed in Section 5, and peers who provided feedback during playtests.

A Repository map and routing

A.1 Programmed routing and shell

Client routes are declared in `client/src/App.tsx`: public splash and auth (`/`, `/auth`, `/auth/admin`); authenticated gameplay and hub routes under `ProtectedRoute` with `Layout` (lobby, poker, blackjack, tournaments, mini-games, profile, friends, admin console). Admin routes use `AdminProtectedRoute`. Global providers stack in `client/src/main.tsx` (error boundary, loader, Redux, toasts, user, socket, Quantum HUD, hidden bets, audio) with an additional nested loader/invitation shell in `App.tsx`. Server REST mounts, rate-limit tiers, and health/metrics endpoints are wired in `server/src/index.ts`. A narrative overview also exists in `docs/ARCHITECTURE.md`.

B Architecture diagram

B.1 Layered runtime architecture

Figure 4 summarizes the logical architecture used by Quantum Bluff. The important design rule is separation of responsibility: the client renders and sends intentions; the backend authenticates, validates and applies rules; domain services execute game and economy logic; PostgreSQL and Redis provide durable state and coordination.

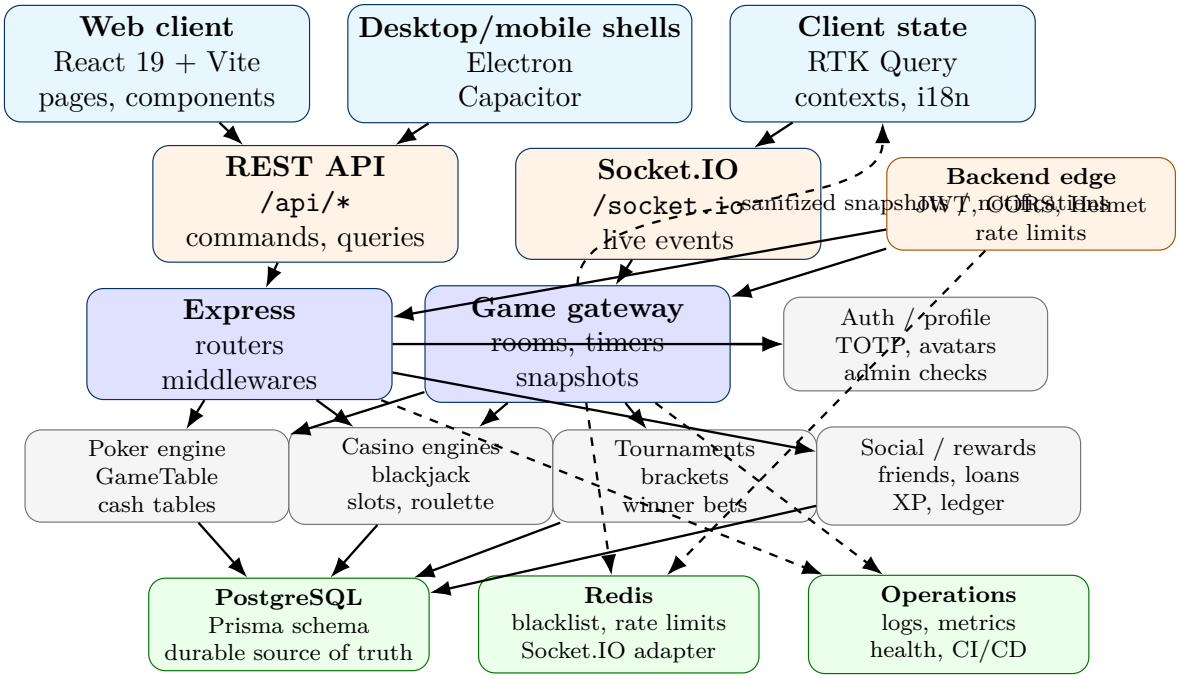

Figure 4: Layered architecture diagram: UI shells and client state, REST/Socket.IO transport, backend edge controls, domain services, PostgreSQL persistence, Redis coordination and operational monitoring.

B.2 Reading the diagram

- **Client layer**: renders the lobby, tables, profiles and modals; it caches state but does not decide outcomes or balances.
- **Transport layer**: REST carries durable commands and queries; Socket.IO carries low-latency gameplay and notification events.
- **Backend edge**: applies JWT identity, role separation, CORS/Helmet, rate limits and request validation before domain code runs.
- **Domain layer**: poker, blackjack, casino, tournament, social and reward services enforce rules and prepare database mutations.
- **Infrastructure layer**: PostgreSQL stores durable state, Redis coordinates short-lived distributed concerns, and operational hooks expose health, metrics and deployment feedback.

C Relational database model

C.1 Relational mode and source of truth

Quantum Bluff uses a **relational PostgreSQL database** accessed through Prisma. The authoritative schema is `server/prisma/schema.prisma`; migrations define the concrete SQL evolution. The central entity is `User`. Most gameplay, wallet, social, tournament and security tables reference it through foreign keys, which keeps identity, chip movements and audit history normalized instead of duplicating user information inside each feature.

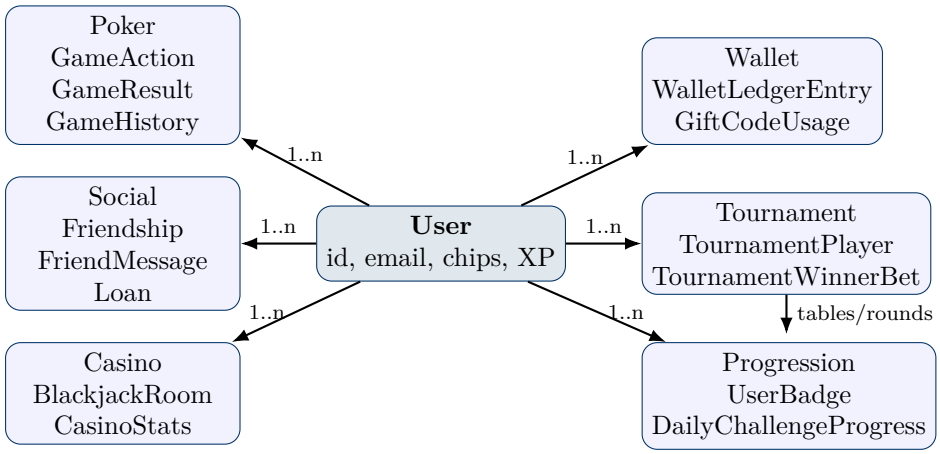

Figure 5: Simplified relational model: `User` is the central parent for gameplay, wallet, social, tournament, casino and progression data.

C.2 Main relational groups

Table 4: Main relational groups in the Prisma schema.

Domain	Main tables	Relation pattern
Identity and profile	`User`, `UserStats`, `PlayerStats`, `CasinoStats`	One user owns optional 1–1 profile/stat rows and many gameplay records.
Poker	`GameHistory`, `GameAction`, `GameResult`, `WaitingRoom`, `RoomPlayer`	Users join rooms, perform actions, and appear in result/history rows.
Tournament	`Tournament`, `TournamentPlayer`, `TournamentRound`, `TournamentTable`, `TournamentWinnerBet`	A tournament has many players, rounds and tables; winner bets link a bettor user to a predicted winner user.
Wallet and rewards	`WalletLedgerEntry`, `TournamentRewardLedger`, `GiftCode`, `GiftCodeUsage`	Each chip-changing operation creates auditable rows tied to a user and reason.

Domain	Main tables	Relation pattern
Social and loans	`Friendship`, `FriendRequest`, `FriendMessage`, `LoanRequest`, `Loan`, `LoanRepayment`	User-to-user relations are modeled through explicit join tables and named Prisma relations.
Casino and blackjack	`BlackjackRoom`, `BlackjackRoomSeat`, `BlackjackRoomSnapshot`, `CasinoStats`	Rooms have seats and snapshots; casino statistics remain attached to the user.
Hidden bets	`HiddenBetTicket`, `HiddenBetSelection`, `HiddenBetHandResolution`	A ticket belongs to a user and game/hand, with child selection rows and resolution records.

D Poker engine and cash game

D.1 What was programmed

Authoritative table logic lives in `server/src/logic/GameTable.ts` (seats, streets, pot and side pots, action legality, showdown ordering) and `server/src/logic/CashGameController.ts` (chip debits/credits, transitions between phases). Hand evaluation and category ordering use `server/src/logic/Evaluator.ts`. Real-time delivery and timers (e.g. `TURN_TIMER` with 20 s auto-fold/check) are implemented in `server/src/sockets/game.gateway.ts` together with anti-cheat hooks. The client implements presentation and input in `client/src/pages/Game.tsx` and reusable components under `client/src/components/` (`PokerTable`, `ActionButtons`, `CommunityCards`, chat, HUD hooks). **All wagering state changes are validated and applied on the server**; the client never "decides" the winner.

D.2 Rules enforced

Texas Hold'em flow from blinds through preflop, flop, turn, river, and showdown; fold, check, call, raise semantics; minimum raise increments; side pots when players are all-in for different amounts. Waiting-room creation and join flows are handled by dedicated routes (`server/src/routes/waitingRoom.routes.ts`, `server/src/routes/game.api.routes.ts`) coordinated with the gateway.

D.3 Further reading

Structured logic notes: `Docs/detail_logique/backend/01_poker_moteur.md`, `Docs/detail_logique/backend/02_cashgame_controller.md`.

E Bot AI and practice integration

E.1 Programmed pipeline

Figures 1 and 2 in the main report depict the optional PyTorch policy ($36 \to 96 \to 64 \to 32 \to 4$, **196** post-input units, **11 972** trainable scalars in the linear stack) and the Node/Python pipeline. This appendix keeps the algebraic detail for heuristics and oracle mode. `POST /api/bot/action` (`server/src/routes/bot.routes.ts`) validates difficulty and cards, then calls `server/src/logic/botAI.ts`. Practice tables chain bot turns through `server/src/poker/services/practiceBotTurns.service.ts`; HTTP orchestration and optional expert delegation sit in `server/src/services/botAi.service.ts`. Four difficulties are supported: `easy`, `medium`, `hard`, `expert`.

E.2 Strength signal and decisions

Let H be the scalar hand score returned by `getHandValue` for the bot's visible cards (two to seven cards). The code normalizes

$$\sigma = \min\Big(1,\, \max(0,\, H/S_{\max})\Big), \qquad S_{\max} = 9 \cdot 15^5 + 14 \cdot 15^4,$$

matching the encoding used by `server/src/logic/Evaluator.ts`. Each difficulty combines σ with pot odds (`callAmount`, `potSize`), heads-up vs. multiway flags, and pseudo-random draws to return `FOLD`, `CALL`, `CHECK`, or `RAISE` with an optional raise amount (chips rounded via `intChips`). The easy tier, for example, injects naive bluffs on a free street with fixed probability and biases calls when facing small bets with medium strength.

E.3 Dispatch surface

`decideBotAction` in `server/src/logic/botAI.ts` switches on `difficulty` and routes to `easyBotDecision`, `mediumBotDecision`, `hardBotDecision`, or `expertBotDecision`. The HTTP layer (`server/src/services/botAi.service.ts`) may intercept `expert` to call an external model first.

E.4 Decision dispatch and bot tier routing

The function `decideBotAction(sigma, potOdds, stackPressure, difficulty)` accepts the normalized hand strength and situation parameters, then switches to the appropriate tier:

```
decideBotAction(sigma, potOdds, stackPressure, difficulty) {
  switch (difficulty) {
    case 'easy': return easyBotDecision(sigma, potOdds);
    case 'medium': return mediumBotDecision(sigma, potOdds, stackPressure);
    case 'hard': return hardBotDecision(sigma, potOdds, stackPressure);
    case 'expert': return expertBotDecision(...);
  }
}
```

Each function returns `FOLD`, `CALL`, `CHECK`, or `RAISE` with an optional raise amount. For the expert tier, the HTTP layer (`server/src/services/botAi.service.ts`) may intercept and call an external model first; on timeout or failure, it reverts to hard tier.

E.5 Bot tier specifics and complexity

Easy tier. Strategy: fixed bluff probability on free streets (check or bet with zero cost to call); calls liberally with $\sigma \geq 0.3$. Decision logic: `if` $\sigma >$ `threshold, return RAISE; else FOLD`. Complexity: $\mathbf{O(1)}$ constant-time decision.

Medium tier. Strategy: integrates pot odds into sigma; compares σ to the call-to-pot ratio `callAmount`/(`potSize` + `callAmount`). Bluff frequency adjusts by position (button more aggressive at $p_{\text{bluff}} = 0.20$; under the gun more conservative at $p = 0.08$). Complexity: $\mathbf{O(1)}$ constant-time.

Hard tier. Strategy: adds opponent hand range tracking; maintains a map of recent player actions (check, call, raise) to infer strength ranges. Decision compares σ against estimated opponent strength distribution via weighted averages. Complexity: $\mathbf{O(n)}$ where $n \leq 8$ is the number of tracked opponents (typical tables have 2–9 players).

Expert tier. Strategy: delegates to external PyTorch model via HTTP POST. Payload includes compact card encoding, current street, stacks, pot size, and position. Fallback to hard tier if service unavailable or timeout (default 2000 ms). Complexity: $\mathbf{O(1)}$ on the bot side (HTTP call is async); external model inference latency is hidden from the decision.

E.6 Expert tier: optional Python service

When `AI_SERVICE_URL` is configured, `botAi.service.ts` issues `POST .../predict/poker` with a JSON payload built by `buildAiPayload` (compact card encoding, street, stacks, pot, position, optional opponent holes for oracle debugging). The JSON body is validated through `aiResponseSchema` (Zod): allowed actions include `ALL_IN`, which is lowered to a legal `RAISE`/`CALL`/`FOLD` via `convertAllIn`. Timeouts use `AbortController` and `env.aiServiceTimeoutMs`; structured logs record `expert_ai_decision` with latency and style. Any failure reverts to `expertBotDecision`.

E.7 Expert oracle path (TypeScript, no network)

For revealed opponent holes, `expertOracleDecision` computes hero strength $\sigma_{\text{hero}} =$ `normalizedHandStreng` and opponent strengths σ_i for each known hole pair. Sort $\sigma_{(1)} \geq \cdots \geq \sigma_{(n)}$. For $n \leq 2$ let $S_{\text{opp}} = \sigma_{(1)}$. For $n \geq 3$ the code uses `compositeOpponentNormalizedStrength`:

$$S_{\text{opp}} = w\,\sigma_{(1)} + (1 - w)\,\sigma_{(\lfloor n/2 \rfloor)}, \qquad w = \min\bigl(0.68,\, \max\bigl(0.35,\, 0.56 - 0.05\,(n - 3)\bigr)\bigr),$$

so the policy blends the strongest visible villain with a median-like opponent instead of assuming everyone holds the nuts. The edge $\Delta = \sigma_{\text{hero}} - S_{\text{opp}}$ is compared to pot odds

`callAmount`/(`potSize` + `callAmount`) and stack pressure `callAmount`/`playerChips` to emit stylised lines (`value`, `bluff`, `pot_control`, `discipline`, etc.).

E.8 Further reading

Design notes: `Docs/IA_EXPERT_BOT.md`.

F Probabilities, Quantum HUD, and Monte Carlo helper

F.1 Division of responsibilities

The **Quantum HUD** (`client/src/components/QuantumHUD.tsx`, context `client/src/contexts/QuantumHUDContext.tsx`) displays *estimates* for the authenticated player. These are computed client-side for responsiveness and are **not** used to settle real-money outcomes (the product uses virtual chips). Server-side hidden-bet pricing uses separate tabular services (Appendix G).

F.2 Monte Carlo equity estimator

Implementation file: `client/src/utils/pokerMonteCarloEquity.ts`. Let N be the number of iterations (default $N = 2000$, constant `MONTE_CARLO_DEFAULT_ITERATIONS`). For each iteration the code:

1. Builds the remaining deck excluding known hero and community cards (`buildDeckExcluding`).
2. Draws, without replacement, the missing board cards and $2 \times n_{\text{opp}}$ opponent hole cards using a partial Fisher–Yates swap sampler (`drawWithoutReplacement`).
3. Evaluates the hero and each opponent seven-card hand with `evaluateSevenEval` (same taxonomy as the server evaluator).
4. Splits the simulated pot equally among all players that achieve the best score.

If $S_t \in \{0, 1/k\}$ is the hero's share on iteration t when k players tie for the best score, the reported equity is

$$\hat{e} = \frac{1}{N} \sum_{t=1}^{N} S_t.$$

A histogram of category indices (0–9) is accumulated in parallel for UX badges.

F.3 Monte Carlo error budget (HUD analogue to RTP)

The HUD does **not** implement a house-banked game: there is no monetary **RTP** in the casino sense. The quantity $\hat{e}$ is an **equity** estimate (expected share of the pot at showdown under uniform completion of unknown cards). If e denotes the true value of that expectation under the model, then $\mathrm{E}[\hat{e}] = e$ (each draw is independent and fair with respect to the remaining deck). Because $0 \le S_t \le 1$,

$$\mathrm{Var}(S_t) \le \frac{1}{4}, \qquad \mathrm{Var}(\hat{e}) = \frac{\mathrm{Var}(S_t)}{N}, \qquad \mathrm{SE}(\hat{e}) \le \frac{1}{2\sqrt{N}}.$$

With the shipped default $N = 2000$, $\mathrm{SE}(\hat{e}) \le 1/(2\sqrt{2000}) \approx 0.01118$ (about 1.1 equity points at most in the worst-case variance bound).

F.4 Quantum Combi—exact enumeration formula

The promotional "Quantum Combi" category (documented for players in `Docs/final_report/QUANTUM_COMBI.adoc`) requires ranks $\{3, 5, 6, 7, 10\}$ each to appear **at least once** among the seven cards (two private + five community). All $\binom{52}{7}$ seven-card subsets are equally likely:

$$\binom{52}{7} = 133784560.$$

For each rank r in that set, let A_r be the event "no card of rank r among the seven"; then $|A_r| = \binom{48}{7}$. By the principle of inclusion–exclusion over five ranks,

$$\Big|\bigcup_r A_r\Big| = \sum_{k=1}^{5} (-1)^{k+1} \binom{5}{k} \binom{52-4k}{7} = 133002736.$$

Hence the favourable count is $133784560 - 133002736 = 781824$ and

$$P(\text{Quantum Combi}) = \frac{781824}{133784560} \approx 0.00584 \approx 0.584\%.$$

F.5 Quantum Combi in the hand-ranking ladder (server and HUD)

Both `server/src/logic/Evaluator.ts` and `client/src/utils/pokerHandCategory.ts` use the same **best-of-seven** cascade. Categories are integers 0 (high card) through 9 (straight flush); a larger packed score beats a smaller one. After ruling out straight flush (9), four-of-a-kind (8), and full house (7), the code tests **Quantum Combi before flush or straight**. Quantum applies iff every rank in $\{3, 5, 6, 7, 10\}$ appears at least once in the seven rank values (Ace is coded as 14). Let V be the multiset of those seven values; subtract one occurrence of each mandatory rank; sort the remaining values descending and take the top two as kickers (κ_1, κ_2). The score is

$$\texttt{packScore}(6, \kappa_1, \kappa_2) = 6 \cdot 15^5 + \kappa_1 \cdot 15^4 + \kappa_2 \cdot 15^3,$$

padding unused kicker slots with zeros in the implementation. Only if Quantum fails does the evaluator continue to flush (5), straight (4), three-of-a-kind, two pair, one pair, high card. Thus Quantum is positioned **strictly between full house and flush**, as a design choice documented in `Docs/final_report/QUANTUM_COMBI.adoc`.

G Hidden betting system

G.1 Programmed architecture

Hidden side markets are implemented under `server/src/poker/hiddenBets/`: typed markets and snapshots (`markets.ts`, `types.ts`), pricing tables (`pricing/pricingTables.ts`, `pricingTables.v1.ts`), live heuristics (`pricing/livePricing.ts`), quote and placement services, ledger context, and street-aware resolvers (`resolver/hiddenBetResolver.ts`). HTTP routes are mounted in `server/src/routes/hiddenBets.routes.ts`.

G.2 Odds and tables (example)

Live markets use tabulated decimal odds keyed by table parameters. For instance, `oddsPlayerWinsCurrentHa` maps the number of active players $n \in [2, 9]$ to a fixed grid (e.g. $n = 2 \to 1.9$, $n = 5 \to 3.0$, $n = 9 \to 4.6$); companion functions cover showdown vs. fold endings. These values are **heuristic** prices distanced from the pre-hand model (see code comments `hidden-bets-live-v1`), not Monte Carlo outputs from the HUD.

G.3 Further reading

`Docs/intern/HIDDEN_BETS_SPEC.md`, `Docs/detail_logique/backend/07_hidden_bets_moteur.md`.

H Tournaments and winner side bets

H.1 Tournament system diagram

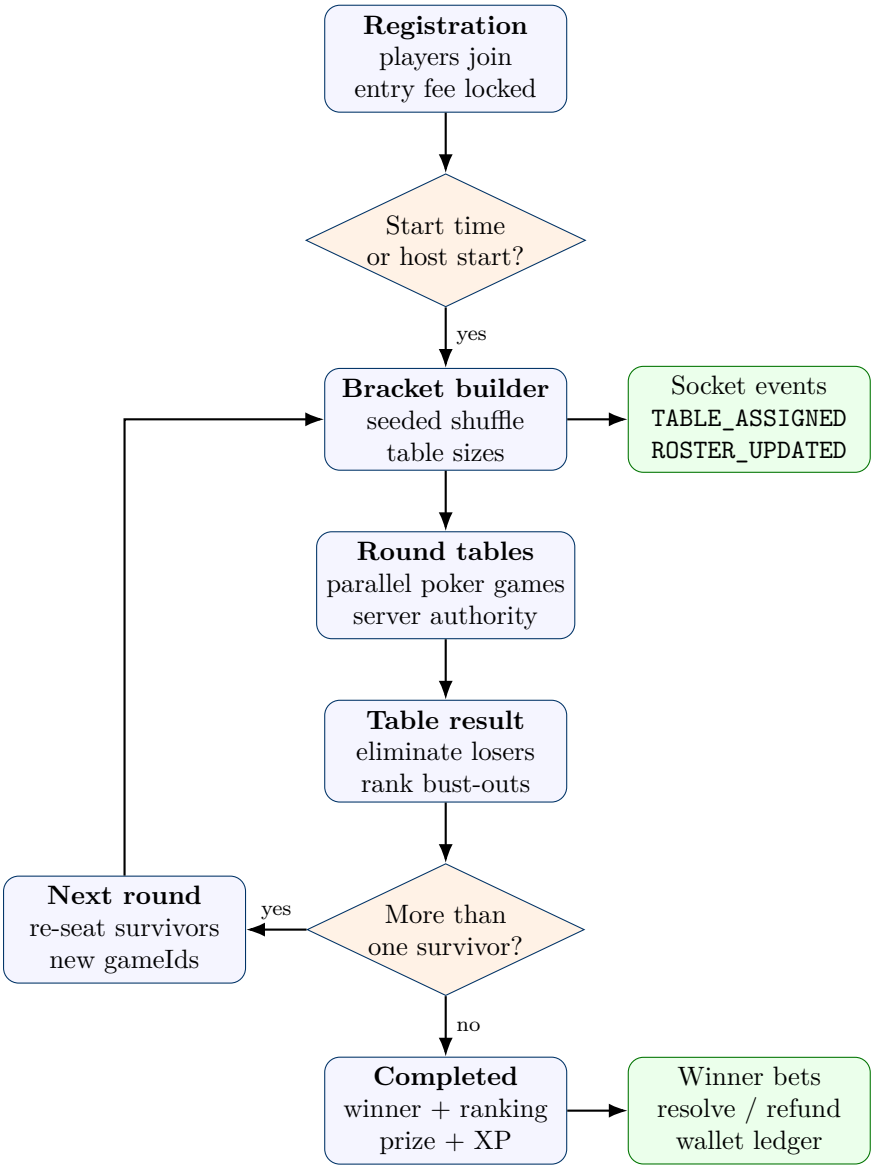

Figure 6: Tournament lifecycle: registration, deterministic bracket generation, parallel poker tables, eliminations, next-round creation, completion and side-bet settlement.

H.2 Poker hand evaluation and tournament algorithms

H.2.1 Algorithm: Mulberry32 seeded PRNG

Tournament bracket reproducibility depends on deterministic pseudorandom number generation. The Mulberry32 algorithm (32-bit integer state) produces uniform random values in $(0, 1)$:

```
export function mulberry32(seed: number): () => number {
```

```
  return function () {
    let t = (seed += 0x6d2b79f5)
    t = Math.imul(t ^ (t >>> 15), t | 1)
    t ^= t + Math.imul(t ^ (t >>> 7), t | 61)
    return ((t ^ (t >>> 14)) >>> 0) / 4294967296
  }
}
```

Identical seeds generate identical sequences, enabling replay and testing. The magic constants (0x6d2b79f5, 0xffffffff masks, rotation widths 7, 15, 14) come from the published Mulberry32 recurrence and ensure good mixing properties across the 32-bit state space.

H.2.2 Algorithm: Fisher–Yates shuffle

Player assignment to tables uses Fisher–Yates shuffling seeded with Mulberry32:

```
export function shuffleWithRng<T>(arr: T[], rng: () => number): T[] {
  const a = [...arr]
  for (let i = a.length - 1; i > 0; i--) {
    const j = Math.floor(rng() * (i + 1))
    ;[a[i], a[j]] = [a[j], a[i]]
  }
  return a
}
```

This produces an unbiased uniform shuffle in $O(n)$ time. Combined with the seeded PRNG, it guarantees that `(playerIds, tournamentSeed)` deterministically yields the same table layout, enabling reproducible testing and replay.

H.2.3 Algorithm: Tournament bracket builder

```
export function buildOpeningRound(playerIds: string[], seed: number) {
  const rng = mulberry32(seed >>> 0)
  const shuffled = shuffleWithRng(playerIds, rng)
  const sizes = computeTableSizesForRound(playerIds.length)
  const tables = []
  let offset = 0
  for (let t = 0; t < sizes.length; t++) {
    tables.push({
      tableIndex: t,
      playerIds: shuffled.slice(offset, offset + sizes[t]),
    })
    offset += sizes[t]
  }
```

```
  return { tables }
}
```

The bracket builder combines seeded randomness with dynamic table sizing to produce fair, reproducible initial assignments. Tables are sized to fit 2–9 players each; for example, 10 players distribute as $[5, 5]$, and 25 players as $[9, 9, 7]$.

H.2.4 Algorithm: Poker hand evaluation

Determining the best five-card hand from seven cards (two private + five community) without brute-force enumeration:

```
export function evaluateHand(sevenCards: Card[]): HandRanking {
  const rankFreq = new Map<number, number>()
  sevenCards.forEach(card => {
    rankFreq.set(card.value, (rankFreq.get(card.value) ?? 0) + 1)
  })
  const suitCounts = sevenCards.reduce((acc, card) => {
    acc[card.suit] = (acc[card.suit] ?? 0) + 1
    return acc
  }, {} as Record<string, number>)
  const isFlush = Object.values(suitCounts).some(count => count >= 5)
  const sortedValues = [...new Set(sevenCards.map(c => c.value))]
    .sort((a, b) => a - b)
  const isStraight = checkConsecutive(sortedValues)
  if (rankFreq.has(4)) return { name: 'Quads', rank: 8 }
  if (rankFreq.has(3) && rankFreq.has(2))
    return { name: 'Full House', rank: 7 }
  if (isFlush) return { name: 'Flush', rank: 6 }
  if (isStraight) return { name: 'Straight', rank: 5 }
  // ... (continues with Three of a kind, Two pair, Pair, High card)
}
```

This O(n) algorithm runs once per hand showdown. By counting rank frequencies and suit occurrences, it avoids the combinatorial explosion of checking all $\binom{7}{5} = 21$ five-card subsets.

H.2.5 Algorithm: Action deduplication cache

Network retransmissions and client retries may send the same action multiple times. Deduplication prevents duplicate chip mutations:

```
class ActionDedupCache {
  private cache: Map<string, unknown> = new Map()
  async process(handId: string, actionId: string,
                processor: () => Promise<unknown>) {
```

```
    const key = `${handId}:${actionId}`
    if (this.cache.has(key)) {
      return this.cache.get(key)
    }
    const result = await processor()
    this.cache.set(key, result)
    return result
  }
}
```

The composite key `handId:actionId` uniquely identifies each player action. A cache hit returns the previous result without re-executing; a miss executes the processor and caches the result. This ensures idempotency: resending the same action produces the same outcome without doubling the effect.

H.2.6 Algorithm: Multi-pot settlement

When players go all-in at different amounts, a single pot is unfair. The algorithm computes side pots:

```
export function settlePots(players: Player[], winnerIds: string[]) {
  const betLevels = [...new Set(
    players.map(p => p.totalPutInThisHand ?? 0)
  )].sort((a, b) => a - b)
  const pots = []
  let prevLevel = 0
  for (const level of betLevels) {
    const increment = level - prevLevel
    pots.push({
      amount: increment * players.length,
      eligibleWinners: winnerIds
    })
    prevLevel = level
  }
  const distribution = {}
  for (const pot of pots) {
    const sharePerWinner = Math.floor(pot.amount / pot.eligibleWinners.length)
    for (const winnerId of pot.eligibleWinners) {
      distribution[winnerId] = (distribution[winnerId] ?? 0) + sharePerWinner
    }
  }
  return distribution
}
```

Example: three players all-in at 100, 150, 200 chips. Bet levels are $\{100, 150, 200\}$. The main pot is $100 \times 3 = 300$ (all three eligible). Side pot 1 is $(150 - 100) \times 3 = 150$ (only the two who contributed 150+). Side pot 2 is $(200 - 150) \times 3 = 150$ (only the one who contributed 200). This ensures no player receives chips they didn't contribute.

H.2.7 Performance and correctness

- Hand evaluation: $O(n)$ per showdown, $n = 7$ cards.
- Bracket generation: $O(m \log m)$ for m players (Fisher–Yates shuffle).
- Action deduplication: $O(1)$ cache lookup.
- Multi-pot settlement: $O(k)$ for k unique bet levels (typically $k \leq 3$).
- Memory: 2 KB per table, deterministic reproducibility via seeded RNG.

H.3 Runtime architecture (layers)

1. **API and validation** — `server/src/routes/tournament.routes.ts` (and related mounts in `server/src/index.ts`) expose creation, join, start, and spectator flows. Input normalisation lives in `tournament.create.validation.ts` (player counts, blind structure, schedule). Private tournaments hash join codes with bcrypt (`tournament.service.ts`).
2. **Persistence** — Prisma models `Tournament`, `TournamentPlayer`, `TournamentRound`, `TournamentTable`, etc., store lifecycle, elimination order, and links to underlying `gameId` values that reuse the cash-game engine.
3. **Scheduler** — `tournament.scheduler.ts` registers a timer every $\Delta T = 10\,\text{s}$ (via `setInterval`). On each tick it loads tournaments with `status = REGISTRATION_OPEN` and `startAt <= now` (bounded batch) and calls `startTournamentFromDb` with the Socket.IO server instance obtained from `app.get('io')`.
4. **Runtime orchestration** — `tournament.runtime.service.ts` coordinates bracket materialisation: it calls `buildOpeningRound` / `buildRoundFromSurvivors`, creates concrete `GameTable` instances through `tournamentTableFactory.ts`, emits `TOURNAMENT_TABLE_ASSIGNED` to per-user rooms, records eliminations when bust-out events arrive (`recordTournamentEliminationsIf` and advances the state machine (`pending_other_tables`, `next_round_spawned`, `tournament_complet`
5. **Winner side bets** — `winnerBets/tournamentWinnerBet.service.ts` handles ticket placement, settlement with tournament cancellation, and coordination with the wallet ledger.
6. **Recovery** — `tournament.recovery.service.ts` reconciles partially written bracket rows after crashes (invoked from server bootstrap).

7. **Gateway hooks** — `tournament.gatewayHook.ts` bridges table-level socket events into tournament bookkeeping.

8. **Client** — `client/src/features/tournament/` implements waiting room, ZipRush, in-tournament HUD, results, and winner-bet UI; routes under `/tournaments/...`.

H.4 Bracket geometry and table sizing

Let m be the number of active players in a round ($m \geq 2$). `computeTableSizesForRound` in `server/src/tournament/bracket/tableSizes.ts` returns an integer partition $[s_1, \ldots, s_k]$ such that $\sum_i s_i = m$, each $2 \leq s_i \leq 9$, using fixed templates for $m \leq 9$ (e.g. $m = 4 \to [2, 2]$, $m = 7 \to [4, 3]$) and, for $m \geq 10$,

$$k = \left\lceil \frac{m}{9} \right\rceil, \qquad s_i \in \left\{ \left\lfloor \frac{m}{k} \right\rfloor, \left\lceil \frac{m}{k} \right\rceil \right\}$$

with the remainder distributed one seat at a time (see implementation). `assertTableSizesValid` enforces $\sum s_i = m$ and per-table bounds.

H.5 Seeded shuffle (deterministic bracket RNG)

`buildOpeningRound` and `buildRoundFromSurvivors` in `TournamentBracketBuilder.ts` take a 32-bit integer `seed`. They derive a PRNG `mulberry32(seed)` producing $u_n \in (0, 1)$ and apply Fisher–Yates shuffling so that the same `(playerIds, seed)` always yields the same table assignment (reproducible tests). The generator follows the published mulberry32 recurrence (32-bit integer state, `Math.imul` mixes, bitwise rotations) as in the source file; the returned uniform is $u = ((t \oplus (t \gg 14)) \gg 0)/2^{32}$.

H.6 Real-time gameplay and tournament dataflow

The tournament orchestration interleaves with concurrent poker games as follows:

1. **Player action**: Client emits `PLAYER_ACTION` (check, bet, fold, call, raise) via WebSocket.

2. **Validation and deduplication**: Backend validates action legality (correct turn, sufficient chips, valid amount). Action deduplication cache (composite key: `handId:actionId`) prevents reprocessing on retransmission.

3. **State mutation**: `GameTable` updates internal state (pot, street progression, elimination status).

4. **Broadcast**: Server broadcasts sanitized `GAME_STATE_SYNC` to all table participants (hiding folded hands from non-showdown participants).

5. **Hand resolution**: At showdown, `evaluateHand` determines the best five-card hand from each player's two private cards plus the five community cards.

6. **Pot settlement**: `settlePots` distributes chips to winners, handling side pots from all-in amounts.

7. **Wallet ledger**: Each chip movement is appended to the immutable `WalletLedgerEntry` table (player ID, reason, amount, balanceBefore/After, SHA-256 integrity hash).
8. **Tournament elimination**: If a player's chip stack reaches zero, they are marked `ELIMINATED` and ranked based on elimination order.
9. **Round advancement**: Once all concurrent tables at a round complete, `tournament.runtime.service.ts` calls `buildRoundFromSurvivors` to seed the next round and reassign survivors to new tables.
10. **Final ranking**: When only one player remains, the tournament transitions to `COMPLETED`, prizes are distributed, and XP is awarded based on final ranking.

This deterministic dataflow ensures consistency: identical player lineups, bet sequences, and RNG seeds always produce identical outcomes, enabling replay, testing, and fairness audits.

H.7 State recovery after crashes

The complete `GameState` snapshot is persisted to PostgreSQL after **every** action via `server/src/shared/pokerStateStore.ts`. The snapshot includes:

- Player stacks and positions.
- Community cards (or empty if pre-flop).
- Current bet amounts and pot.
- Action history (sequence of checks, bets, folds).
- Elimination order (if any).
- Deduplication cache (to prevent re-execution of actions).

On server restart or crash:

1. **Bootstrap**: `tournament.recovery.service.ts` loads the latest snapshot from PostgreSQL.
2. **Replay**: Queued actions (e.g., in Redis or in-memory buffer) are replayed from the snapshot state in order, using the same hand evaluation, deduplication logic, and deterministic RNG.
3. **Validation**: After replay, the server verifies that reconstructed balances match ledger checksums (SHA-256 integrity hashes).
4. **Resume**: The tournament resumes from the exact state it left off, transparent to players.

Deterministic RNG, deduplication caching, and full snapshot persistence together guarantee that the same sequence of player actions always produces the same chip distribution and ledger entries, enabling reliable recovery from any failure point.

H.8 Further reading

`Docs/intern/TOURNAMENT_FLOW_REAL_VS_EXPECTED.md`.

I Real-Time Communication and WebSocket Architecture

I.1 JWT authentication middleware and token validation flow

Socket authentication occurs in `server/src/middleware/socketAuth.middleware.ts` during the Socket.IO handshake. The middleware extracts the JWT from either `socket.handshake.auth.token` or from the `Authorization: Bearer ...` header:

```
io.use((socket: AuthenticatedSocket, next) => {
  const authToken = socket.handshake.auth?.token
  const headerAuth = socket.handshake.headers?.authorization
  const token = authToken ||
    (typeof headerAuth === 'string' ? headerAuth.split(' ')[1] : undefined)

  if (!token) return next(new Error('Token manquant'))

  if (await isBlacklisted(token))
    return next(new Error('Token révoqué'))

  const decoded = verifyToken(token)

  if (decoded.role === 'admin')
    return next(new Error('Token joueur requis'))

  const user = await prisma.user.findUnique({
    where: { id: decoded.userId },
    select: { id: true, bannedUntil: true }
  })

  if (!user) return next(new Error('Session expirée'))
  if (user.bannedUntil && user.bannedUntil > new Date())
    return next(new Error('Compte suspendu'))

  socket.userId = decoded.userId
  next()
})
```

This checklist ensures that:

1. Missing tokens are rejected before JWT verification.
2. Revoked tokens (e.g., after logout or password reset) deny access.
3. Admin tokens cannot be used for regular gameplay (privilege separation).
4. User records are still valid in the database.

5. The user account is not under suspension.

6. The socket is enriched with the authenticated `userId`.

Once authenticated, the socket joins a private user room `user:{userId}` and the user is marked online via `server/src/services/presence.service.ts`. Subsequent user presence changes (activity updates, login/logout) trigger `FRIEND_STATUS_CHANGED` broadcasts to all online friends.

I.2 Room organization and state broadcasting patterns

Socket.IO rooms segment players, spectators, and administrative channels. The primary room types are:

User room (`user:{userId}`): Each authenticated user joins exactly one private room. Events emitted here include tournament assignments, friend requests, loan notifications, and presence updates. This ensures targeted unicast delivery without exposing events to other players.

Game table room (`gameId`): All players and spectators at a poker or blackjack table join this room. When a player takes an action (bet, check, fold), the server broadcasts the updated game state to all room participants via:

```
io.to(gameId).emit('GAME_STATE_UPDATED', sanitizedSnapshot)
```

The snapshot is computed differently for each participant: seated players see their own hole cards, spectators see empty hand arrays, and opponents see folded hands as unavailable.

Waiting room (`roomId`): Lobbies where players select stakes and await game start. Room visibility (PUBLIC, PRIVATE, FRIENDS_ONLY) is enforced: unauthorized users cannot join even if they craft a socket request.

Tournament rooms (`tournament:{tournamentId}`, `TOURNAMENT_LOBBY`): Used for bracket announcements, round transitions, elimination notifications, and winner-bet settlements. Players can subscribe to specific tournaments they are registered in; the TOURNAMENT_LOBBY broadcasts global roster updates.

Broadcasting is **idempotent and safe**: if a client misses an event due to temporary disconnection, the next full snapshot will resynchronize state. Legacy event names (`GAME_UPDATE`) and current names (`GAME_STATE_UPDATED`) are both emitted during transitions to maintain backward compatibility.

I.3 Snapshot synchronization and disconnection resilience

State recovery after temporary disconnection or page refresh relies on three mechanisms:

I.3.1 In-memory runtime state

Active games are stored in a runtime registry (`server/src/shared/activeGames.ts`), mapping `gameId` → `GameTable` or `CashGameController`. Each controller maintains the current hand state (player stacks, community cards, pot, action history). On reconnect, the gateway queries this registry to retrieve the latest state without database round-trips.

I.3.2 Serializable snapshots

For durability, snapshots are also pushed to a persistent store (`server/src/shared/pokerStateStore.ts`). These snapshots record the complete game state after every action, enabling recovery if the server process restarts:

1. Client sends action (e.g., bet 100 chips).
2. Server validates action and updates runtime state.
3. Server writes snapshot to PostgreSQL.
4. Server broadcasts updated snapshot to all room participants.
5. On reconnect, server loads the latest snapshot and resumes play.

I.3.3 Full snapshot on join/reconnect

Instead of relying on incremental event playback, the gateway immediately sends a full sanitized snapshot when a socket joins or reconnects:

```
const snapshot = game.getSanitizedState(requestingUserId, isSpectator)
socket.emit('GAME_UPDATE', snapshot)
socket.emit('GAME_STATE_UPDATED', snapshot)
```

This ensures that any queued or missed events are superseded by a complete state refresh.

I.4 Table locking and action serialization

Concurrent player actions at the same table can cause race conditions (e.g., two players raising simultaneously, or a player betting and folding in the same tick). To prevent this, the gateway uses fine-grained table locks:

```
await withPokerTableLock(gameId, `cashbet:${userId}`, async () => {
  await applyPokerAction({
    gameId,
    userId,
    action: 'RAISE',
    amount: 50
  })
})
```

The lock key is `{gameId}:{action_type}:{userId}`. While a lock is held:

1. All other actions on that game are queued.
2. The current action executes atomically: state update + broadcast + ledger write.
3. After release, the next queued action proceeds.

This serialization ensures that chip mutations are atomic and ordered, preventing double-betting or race-condition exploits. Lock timeouts (e.g., 5 seconds) guard against deadlock if a handler crashes.

I.5 Anti-cheat monitoring and action deduplication

Player actions may be retransmitted due to network jitter or client retry logic. To prevent duplicate mutations, the gateway maintains an **action deduplication cache**:

```
class ActionDedupCache {
  private cache: Map<string, unknown> = new Map()

  async process(handId: string, actionId: string,
                processor: () => Promise<unknown>) {
    const key = `${handId}:${actionId}`
    if (this.cache.has(key))
      return this.cache.get(key)  // Cache hit: return previous result

    const result = await processor()
    this.cache.set(key, result)    // Cache miss: execute & store
    return result
  }
}
```

If a client resends the same action (same handId and actionId), the cache returns the previous outcome without re-executing the processor. This is crucial for ensuring idempotency: resending a bet does not cause double-deduction.

The `AntiCheatMonitor` (in `server/src/utils/antiCheat.ts`) also tracks suspicious patterns:

- **Rapid action spam**: > 8 actions within 3000 ms triggers a warning.
- **Turn timer violations**: if a player acts before the turn timer expires, or acts after a timeout, it is logged.
- **Impossible action combinations**: e.g., checking and then raising in the same action sequence.
- **Invalid amounts**: bets exceeding stack size or non-positive amounts.

Flagged actions are logged via `server/src/utils/securityLogger.js` for review; serious violations may trigger account suspension.

I.6 Redis pub/sub adapter for multi-instance deployments

When multiple server instances run behind a load balancer, Socket.IO alone cannot broadcast a room message from instance A to sockets connected to instance B. Redis bridges this gap:

I.6.1 Single instance (development fallback)

Without Redis, broadcasts reach only sockets on the current instance. Suitable for local development but insufficient for production.

I.6.2 Multi-instance with Redis

The Socket.IO Redis adapter (`server/src/config/socketIoRedis.ts`) creates dedicated pub/sub clients:

```
export function createSocketIoRedisClients() {
  const pubClient = new Redis(redisUrl, {
    retryStrategy: (times) => Math.min(times * 50, 2000),
    maxRetriesPerRequest: 20
  })
  const subClient = pubClient.duplicate()

  return { pubClient, subClient }
}
```

When instance A broadcasts to room `tournament:abc`, the Redis adapter:

1. Publishes a message to the Redis channel `socket.io_tournament:abc`.
2. All instances (A, B, C, ...) subscribe to this channel.
3. Each instance forwards the message to its local sockets in that room.

This enables seamless horizontal scaling: new instances can join the cluster without reconfiguring existing ones. Redis connection failures are handled gracefully: if Redis is unavailable, the application reverts to local-only broadcasting, preserving availability for existing clients.

I.7 Further reading

`server/src/sockets/game.gateway.ts`, `server/src/middleware/socketAuth.middleware.ts`, `server/src/config/socketIoRedis.ts`, `Docs/ARCHITECTURE.md`.

J Blackjack solo and multiplayer

J.1 Solo blackjack engine

Authoritative rules are implemented in `server/src/logic/blackjack.ts`. The server uses a six-deck shoe (`DECKS = 6`), shuffled in-place with Fisher–Yates (`shuffleShoe`). Card values follow standard rules: faces and tens count as 10; aces count as 11 when possible without exceeding 21, otherwise as 1. The dealer **stands on soft 17** (e.g. ace-six totaling 17) and draws on hard 17 and below. Natural blackjack (ace plus ten-value card dealt as the initial hand) pays 3:2 on the stake; all other wins pay 1:1; pushes (tie) return the stake to the player. Double down is permitted on exactly two player cards and issues one additional card; no further draws after doubling. No surrender option.

J.1.1 Payout structure and expected value

Let the player's stake be B chips. Outcomes and payouts:

- **Natural blackjack**: player receives stake B plus winnings $1.5B$ (total return: $2.5B$).
- **Regular win**: player receives stake B plus winnings B (total return: $2B$).
- **Push**: player receives stake B (total return: B).
- **Loss**: player receives 0 (total return: 0).

Under proper basic strategy (not enforced by the server; the player may make suboptimal plays), the expected return to player is approximately 99.5% against a single-deck game. With the server's six-deck shoe and no card counting, realistic RTP is approximately 99.3%. The exact value depends on the player's strategy deviations and bet sequencing.

J.1.2 State and recovery

Game state (dealer hand, player total, bet amount, action history) is persisted to PostgreSQL after each action via `server/src/blackjack/recovery/blackjackSnapshot.repository.ts`. In case of crash, the system reloads the snapshot and replays to restore the exact table state.

J.2 HTTP and multi-table surfaces

Solo routes: `server/src/routes/blackjack.routes.ts`. Multiplayer rooms and seats: `server/src/routes/blackjackMulti.routes.ts`, controllers under `server/src/logic/BlackjackTableC ts`, in-memory runtime maps with recovery under `server/src/blackjack/`. UI: `client/src/pages/Blackjack.tsx`, lobby/table flows, components in `client/src/components/blackjack/`.

J.3 Further reading

Docs/detail_logique/backend/03_blackjack_solo_moteur.md, Docs/detail_logique/backend/04_blackjack_multi_moteur.md.

K Slot machine, roulette, and casino wallet

K.1 Three-reel slot model

The server module `server/src/logic/slotMachine.ts` draws three independent reels. Symbol weights are

$$w_{\text{cherry}} = 32,\ w_{\text{lemon}} = 24,\ w_{\text{bell}} = 18,\ w_{\text{seven}} = 14,\ w_{\text{diamond}} = 8, \qquad W = \sum_i w_i = 96.$$

One reel satisfies $\Pr[R{=}i] = w_i/W$. Triples satisfy $\Pr(R_1{=}R_2{=}R_3{=}s) = (w_s/W)^3$. Triple s pays total return multiplier $m_s \in \{5, 8, 10, 15, 20\}$ on the stake; an **exact pair** (two equal symbols, third different) pays multiplier 1 (stake returned, zero net profit); otherwise the payout multiplier is 0. RNG is injectable (`RandomIntFn`, default `crypto.randomInt` via `server/src/rng/rng.service.ts`).

K.2 Complete slot RTP (closed form, current constants)

Let $X =$ `winAmount/bet` be the **gross return multiplier** (total coins returned by the machine, including the stake on a push). Write R_1, R_2, R_3 for the three reel symbols.

Triple (three of a kind).

$$\Pr(\text{triple of } s) = \Big(\frac{w_s}{W}\Big)^3, \qquad \Pr(\text{any triple}) = \sum_s \Big(\frac{w_s}{W}\Big)^3 = \frac{\sum_s w_s^3}{W^3}.$$

Numerically $\sum_s w_s^3 = 32768 + 13824 + 5832 + 2744 + 512 = 55680$ and $W^3 = 96^3 = 884736$, hence

$$\Pr(\text{any triple}) = \frac{55680}{884736} \approx 0.0629\ (6.29\%).$$

Exact pair (not a triple). For a doubled symbol A,

$$\Pr(\text{pair with } A \text{ doubled}) = \frac{3\,w_A^2\,(W - w_A)}{W^3}, \qquad \Pr(\text{any pair}) = \sum_A \frac{3\,w_A^2\,(W - w_A)}{W^3}.$$

Numerically $\sum_A 3w_A^2(W - w_A) = 461952$, so $\Pr(\text{pair}) = 461952/884736 \approx 0.5221$ (52.21%).

Lose.

$$\Pr(\text{lose}) = 1 - \Pr(\text{triple}) - \Pr(\text{pair}) \approx 0.4149\ (41.49\%).$$

Expected multiplier and RTP. Because outcomes partition $\{\text{lose}, \text{pair}, \text{triples}\}$,

$$\mathrm{E}[X] = \sum_s \Big(\frac{w_s}{W}\Big)^3 m_s + \Pr(\text{pair}) \cdot 1 = \frac{\sum_s m_s w_s^3 + 461952}{884736}.$$

Here $\sum_s m_s w_s^3 = 5 \cdot 32768 + 8 \cdot 13824 + 10 \cdot 5832 + 15 \cdot 2744 + 20 \cdot 512 = 384152$, so

$$\mathrm{E}[X] = \frac{384152 + 461952}{884736} = \frac{846104}{884736} \approx 0.9563.$$

The **return-to-player (RTP)** expressed as a percentage of the stake is $100\,\mathrm{E}[X] \approx 95.63\%$. The **house edge** on the stake is $1 - \mathrm{E}[X] \approx 4.37\%$. The expected **net** gain per spin is $\mathrm{E}[G] = \mathrm{E}[\texttt{winAmount} - \texttt{bet}] = \texttt{bet}\,(\mathrm{E}[X] - 1) \approx -0.0437 \times \texttt{bet}$.

Variance (optional diagnostic). With the same X, the AsciiDoc reference reports $\mathrm{Var}(X) \approx 3.12$ (standard deviation ≈ 1.77); hence $\mathrm{Var}(G) = \texttt{bet}^2\,\mathrm{Var}(X)$ for a flat bet. See `Docs/SLOT_MACHINE.adoc` for the full derivation.

K.3 European roulette model

The server module `server/src/logic/roulette.ts` draws a single winning integer uniformly from $\{0, \ldots, 36\}$ (`crypto.randomInt` through the same RNG service). Payouts follow the European **total-return** multipliers on the stake for winning pockets (straight 36, split 18, street 12, corner 9, six-line 6, dozen/column 3, even-money 2). Zero causes outside bets to lose unless covered.

K.4 Complete roulette RTP (all proportional inside bets)

Consider any bet line that covers exactly k distinct winning pockets ($1 \le k \le 18$ on the proportional ladder used in code) with European gross multiplier $m = 36/k$ applied to the stake when one of those pockets hits. Because each pocket has probability $1/37$,

$$\mathrm{E}\Big[\frac{\texttt{payout}}{\texttt{bet}}\Big] = \frac{k}{37} \cdot m = \frac{k}{37} \cdot \frac{36}{k} = \frac{36}{37} \approx 0.97297.$$

Hence every such line has the same **RTP** $= 36/37 \approx 97.297\%$ of the stake returned as gross payout in expectation, and the same **house edge** $= 1/37 \approx 2.703\%$ on the stake. The expected **net** gain per unit staked on a single line is

$$\frac{k}{37} \cdot m - 1 = -\frac{1}{37}.$$

When several independent bet lines are placed on one spin, expectation adds across lines; the server enforces caps (`ROULETTE_MAX_BET_CAP`, `ROULETTE_MAX_TOTAL_STAKE`, `ROULETTE_MAX_BETS_PER_SPIN`) before settlement. Wheel animation order is `EUROPEAN_WHEEL_ORDER`. Full payout tables: `Docs/final_report/ROULETTE.adoc`.

K.5 Wallet and ledger

Casino flows integrate with wallet/ledger helpers under `server/src/casino/` and `server/src/wallet/`. Additional narrative: `Docs/detail_logique/backend/05_roulette_moteur.md`, `Docs/detail_logique/backend/06_slot_moteur.md`, `Docs/final_report/ROULETTE.adoc`.

K.6 RNG injection and deterministic testing

All casino games use the RNG service (`server/src/rng/rng.service.ts`) for random draws. In production, the default is `crypto.randomInt`, providing cryptographic entropy from the OS. For testing and audit purposes, RNG is injectable: callers may pass a mock deterministic function (e.g. a seeded Mersenne Twister or constant sequence) to verify game logic without randomness. This allows unit tests to assert exact payout sequences and simplifies debugging of edge cases.

K.7 Ledger integration and fairness guarantees

Every casino outcome (slot spin, roulette pocket, blackjack result) is recorded atomically in the wallet ledger (`server/src/casino/services/walletLedger.service.ts`) with an immutable entry tagged `GAME_WIN` or `GAME_LOSS`, including:

- `userId`: the player
- `reason`: e.g. `SLOT_WIN`, `ROULETTE_LOSS`
- `balanceBefore`, `balanceAfter`: chip ledger state
- `amount`: net gain or loss in chips
- `gameType`: `'casino'`
- `roundId`: unique game identifier (spin ID / hand ID)
- `createdAt`: ISO timestamp

All ledger updates are wrapped in Prisma transactions; balance updates and ledger inserts occur atomically or not at all. This immutable audit trail enables forensic analysis, chargeback investigation, and regulatory compliance.

K.8 Anti-cheat and outcome validation

The server validates outcome feasibility before crediting:

- Slot bets must satisfy $\texttt{SLOT\_MIN\_BET} \leq \texttt{bet} \leq \texttt{SLOT\_MAX\_BET}$.
- Roulette bets must respect `ROULETTE_MAX_BET_CAP` per line and `ROULETTE_MAX_TOTAL_STAKE` per spin.
- Blackjack bets must be consistent between initial bet and any double-down.

Client-side display receives only the outcome, never the pre-draw state or seed; clients cannot manipulate outcomes. WebSocket messages are validated and timestamped server-side; replay attacks are mitigated by idempotent action IDs.

L Social graph, invitations, and peer loans

L.1 Programmed surfaces

Friends UI: `client/src/pages/Friends.tsx`. REST: `server/src/routes/friends.routes.ts`; poker invitations `server/src/routes/invitation.routes.ts` (also under `/api/invitations`); blackjack room invitations share patterns with the lobby. Peer loans: `server/src/routes/friendLoan.routes.ts`, business logic in `server/src/services/friendLoan.service.ts`, real-time updates via `server/src/services/friendLoan.emit.ts` and socket listeners on the client.

L.2 Further reading

`Docs/detail_logique/backend/09_friend_loans_moteur.md`.

M Progression, challenges, and leaderboards

M.1 Programmed logic

Shared rules in `server/src/logic/gamification.ts` (XP curves, badge triggers). Daily challenges and login streak are isolated modules (`server/src/dailyChallenges/`, `server/src/dailyLogin/`) with dedicated routers. Public leaderboards: `server/src/routes/leaderboard.routes.ts`; client pages under `/leaderboard` and profile summaries.

M.2 Further reading

`Docs/detail_logique/backend/10_gamification_moteur.md`.

N Rewards, Banking, and Transactions

N.1 Gamification: XP curves, levels, and badge unlocks

The core XP formula is:

$$\text{XP threshold for level } L = 50 \cdot L \cdot (L-1), \quad L \in \{1, 2, \ldots, 99\}$$

Implementation in `server/src/logic/gamification.ts`. XP awards per game mode: poker (hand vs. bot: 12 XP + 20 bonus on win; showdown vs. human: 35 XP win / 10 loss), blackjack (5 XP + 10 bonus on win), roulette/slots (4 XP + 8–10 bonus on win), tournaments (500 / 400 / 300 / 50 XP per placement). Badge unlocks are level-driven: 10 tiers (novice→legend) defined in `BADGE_CATALOG`. Level-gated betting limits: slot max bet scales from 500 to 5000 chips (capped at level 25); blackjack fixed at 1000; roulette per-line at 500, total stake 3000. Enforcement functions: `getEffectiveSlotMaxBet`, `getEffectiveBlackjackMaxBet`, `getEffectiveRouletteMaxPerLine`.

N.2 Wallet ledger: immutable transaction record

Every chip movement (game win/loss, loan disbursement, tournament prize) creates an immutable `WalletLedgerEntry` row in PostgreSQL (`server/src/casino/services/walletLedger.service.ts`). Fields: userId, reason (e.g. `GAME_WIN`, `TOURNAMENT_PRIZE`, `LOAN_REPAYMENT_IN`), amount, balanceBefore / balanceAfter, roundId, actionId, gameType, integrityHash (SHA-256 of all fields + version numbers), settlementState (`SETTLED` or `PENDING`), createdAt. All ledger updates are wrapped in Prisma transactions; balance updates and ledger inserts occur atomically or not at all. Integrity hash allows forensic verification: any tampering invalidates the hash.

N.3 Tournament rewards and peer loans

Tournament prize pools sum all entry fees (typically 10% of initial stack). Winner receives the pool as a single ledger entry (`TOURNAMENT_PRIZE`); all participants receive placement-based XP. Reward grants are idempotent via `TournamentRewardLedger` unique constraint (`server/src/tournament/tournament.reward.service.ts`), enabling crash recovery. Peer-to-peer loans between friends specify principal P and repayment rate $r\% \in \{10, 15, 20, 25, 30, 40, 50\}$ (see `server/src/logic/friendLoan.interest.ts`). System converts r to annual interest via lookup table: 10%→30%, 15%→24%, 20%→18%, 25%→14%, 30%→10%, 40%→7%, 50%→5%. When borrower wins chips, server automatically diverts slice to lender: slice = $\min(\text{remaining\_due}, \lfloor \text{grossWin} \cdot (r/100) \rfloor)$. Repayment is mandatory; loan settles atomically. Constraints (enforced in `server/src/services/friendLoan.service.ts`): borrower and lender must be friends, borrower can have at most one active loan, lender must have sufficient chips at acceptance, max one pending request between same pair, rate-limited to 40 operations per 10 minutes.

N.4 Further reading

`Docs/detail_logique/backend/10_gamification_moteur.md` (XP and badge mechanics). `server/src/logic/friendLoan.interest.ts`, `server/src/routes/friendLoan.routes.ts`, `server/src/services/friendLoan.service.ts` (peer loan implementation).

O Authentication, profiles, and i18n

O.1 Programmed security and profiles

JWT issuance and verification on `server/src/routes/auth.routes.ts`; password hashing (bcrypt) and validation schemas (Zod). Optional TOTP 2FA: `server/src/routes/twofa.routes.ts`. Client auth flows: `client/src/pages/Auth.tsx`. Profile and avatar uploads: `client/src/pages/EditProfile.tsx` with binary fields or URLs on the Prisma `User` model. Internationalization bundles live in `client/src/i18n/locales/` (five languages); guest-only screens force English overlays where documented in Section 11.

P Operations, Redis, CI, and packaging

P.1 Programmed operations

Redis clients for Socket.IO horizontal scale-out: `server/src/config/socketIoRedis.ts`. Rate limiting and action logs share Redis primitives configured in `server/src/index.ts`. Observability helpers: `server/src/observability/`. CI pipeline: `.gitlab-ci.yml`. Admin web console: `client/src/pages/AdminConsole.tsx` with `server/src/routes/adminConsole.routes.ts`. Dangerous admin/dev routes are compiled out or gated when `env.isProduction` is true.

P.2 Kaniko container builds (GitLab CI)

The `docker-build` stage invokes **Kaniko** executor `v1.23.2-debug` with an empty container `entrypoint` so the shell can call `/kaniko/executor`. **(1)** `docker:build`—`server/Dockerfile`, context `server/`, destinations `${CI_REGISTRY_IMAGE}:${CI_COMMIT_SHORT_SHA}` and `:latest`; runs on `develop`, `main`, and `release/*`. **(2)** `docker:build:client`—`client/Dockerfile`, build-arg `VITE_API_URL` (override in GitLab CI/CD variables for behind-nginx paths), `KUBERNETES_MEMORY_REQUEST`/`LIMIT` raised to reduce OOM during Kaniko layer snapshots; destinations `:client-${CI_COMMIT_SHORT_SHA}` and `:client-latest`; same branch rules as (1). **(3)** `docker:build:ai-service`—context `server/ai-service/`, FastAPI stack for the optional poker prediction endpoint; destinations `:ai-${CI_COMMIT_SHORT_SHA}` and `:ai-latest`; **only** `release/*` (commented in YAML to spare runner disk on `develop`/`main`). All three jobs use `-compressed-caching=false` for predictable memory on small Kubernetes runners. `client/Dockerfile` documents Kaniko-oriented Dockerfile layering (grouped `RUN` steps) to avoid oversized intermediate snapshots after `npm ci`.

P.3 Electron and Capacitor build matrix

The npm scripts are declared in `client/package.json` (see `build` block for electron-builder). `npm run build:electron` runs Vite in `electron` mode; `electron:build:win`, `electron:build:mac`, `electron:build:linux`, and `electron:build:all` invoke `electron-builder` with the corresponding platform flags. Windows uses NSIS (`.exe` installer); macOS uses the default mac target (typically `.dmg` plus update artefacts); Linux emits `AppImage` and `.deb` as declared under `build.linux.target`. `publish:win` / `publish:mac` add `-publish always` for CI/CD uploads to the generic update server configured in `build.publish`.

Capacitor: `npm run build:cap` then `npx @capacitor/cli sync` copies web assets into `android/` and `ios/` native shells (`@capacitor/android`, `@capacitor/ios` 8.3.0). `npm run cap:android` / `cap:ios` open the native IDEs; `cap:run:*` supports live-reload against a LAN API host for device testing.

Q Test coverage metrics and definitions

Q.1 Jest ratios

Let L be the set of executable lines inside `collectCoverageFrom` in `server/jest.config.cjs`. For each metric $M \in \{\text{lines}, \text{statements}, \text{functions}, \text{branches}\}$,

$$\text{cov}_M = \frac{\#\text{ elements of } M \text{ hit during tests}}{\#\text{ elements of } M \text{ instrumented}}.$$

CI enforces $\text{cov}_{\text{lines}} \geq 0.80$, $\text{cov}_{\text{statements}} \geq 0.80$, $\text{cov}_{\text{functions}} \geq 0.80$, and $\text{cov}_{\text{branches}} \geq 0.65$. A May 2026 local run reported $(\text{cov}_{\text{lines}}, \text{cov}_{\text{statements}}, \text{cov}_{\text{functions}}, \text{cov}_{\text{branches}}) = (86.56\%, 83.69\%, 88.75\%, 67.90\%)$ on that scope. Heavy files such as `server/src/logic/botAI.ts` and large controllers are deliberately excluded from the denominator so the metric tracks testable business modules (see comments in the Jest config).

Q.2 Vitest ratios (client)

Vitest with Istanbul uses the same hit/total idea over `client/src/**/*.ts,tsx` minus tests (see `client/vite.config.ts`). Because most screens depend on sockets and browser APIs, the **global** line coverage stays low unless many components receive dedicated tests; quality is also asserted through Playwright flows and manual regression lists.

Q.3 Relation to tournaments

Bracket helpers (`tableSizes.ts`, `TournamentBracketBuilder.ts`) sit inside the Jest coverage glob; their golden tests contribute directly to the aggregate in Table 3. See Appendix H for the mathematics they implement.

Q.4 Further reading

`Docs/test_coverage.md`, `Docs/TESTING_GUIDE.md`, `Docs/RAPPORT_TESTS.md`.

R Real-time, security, and implemented solution details

R.1 Real-time communication details

Real-time communication is implemented with Socket.IO on the `/socket.io` path. The design follows an authoritative server model: the browser can request an action, but the server decides whether the user is authenticated, whether the action is legal, how chips move, and which sanitized state each participant may receive. This is essential for poker because private cards, turn order, pot resolution and hidden side bets must never be trusted to the client.

The socket handshake uses the same identity model as the REST API. A player token is sent through `handshake.auth.token` or an `Authorization: Bearer ...` header, then validated by `server/src/middleware/socketAuth.middleware.ts` and gateway-level checks in `server/src/sockets/game.gateway.ts`. The server verifies token revocation, JWT issuer/audience/type, role separation, user existence and suspension status before attaching the authenticated user id to the socket.

Socket.IO rooms organize delivery. Each user joins `user:{userId}` for targeted events such as invitations, tournament assignments, blocked-user warnings and social notifications. Poker and blackjack tables use table rooms; tournaments use rooms such as `tournament:{id}`. This prevents unrelated users from receiving game or social events that do not concern them.

Game synchronization combines incremental events with full snapshots. Poker runtime controllers are tracked through `server/src/shared/activeGames.ts`; serializable poker snapshots are stored through `server/src/shared/pokerStateStore.ts` and produced by `server/src/poker/services/pokerStateSync.service.ts`. Blackjack recovery and snapshot logic lives under `server/src/blackjack/recovery/` and `server/src/blackjack/services/`. On join or reconnect, the client receives a full sanitized snapshot so a refresh or temporary disconnection does not leave the UI in a stale state.

Sanitization protects confidentiality. A seated poker player can receive their own private cards, while other players and spectators receive only public table information. Redis support through `server/src/config/socketIoRedis.ts` allows Socket.IO events to propagate across multiple server instances.

R.2 Authentication and anti-cheat details

Authentication is based on short-lived JWT access tokens generated by `server/src/auth/jwt.service.ts`. Tokens must use the expected issuer, audience and `type: access`. HTTP routes accept only the standard `Authorization: Bearer ...` form, and protected routes use `server/src/middleware/auth.middleware.ts` to reject revoked tokens, invalid tokens, admin-role tokens on player routes, missing users and suspended accounts.

Passwords are hashed with bcrypt in `server/src/routes/auth.routes.ts`. Registration, reset and profile password changes use Zod validation from `server/src/validation/auth.validation.ts`, including complexity requirements. Optional TOTP two-factor authentication is implemented in `server/src/routes/twofa.routes.ts`; setup secrets are stored temporarily in Redis, and disabling 2FA requires both the password and a valid one-time

code. Logout and token revocation use `server/src/auth/tokenBlacklist.ts`, which stores hashed token entries with expiration.

The same identity policy applies to WebSockets. `server/src/middleware/socketAuth.middleware.ts` rejects revoked, invalid, admin, missing-user and suspended-user tokens before a socket can interact with game rooms. In `server/src/sockets/game.gateway.ts`, sensitive events compare `socket.userId` with any submitted player id; mismatches are refused and logged as suspicious.

Anti-cheat is layered. `server/src/middleware/antiCheat.middleware.ts` blocks active bans, records the last observed IP address and delegates suspicious-pattern detection to `server/src/services/antiCheat.service.ts`. The service can raise alerts for multi-account patterns and unrealistically fast reactions. At game level, `server/src/utils/antiCheat.ts` tracks action bursts and helps refuse excessive gameplay actions.

Operational safeguards complete the baseline: Helmet and CORS allowlists in `server/src/index.ts`, segmented rate limits for sensitive endpoints, idempotency middleware for sensitive mutations in `server/src/middleware/idempotency.middleware.ts`, separated admin routes, blocked-user filtering, blocked-room warnings and persisted player reports for moderation review.

R.3 Implemented solution details

The project is organized into domain modules sharing one runtime architecture: React/TypeScript client, Express REST API, Socket.IO gateway, Prisma/PostgreSQL persistence and Redis coordination. The client renders state and gathers user input; the server validates rules, computes outcomes, applies wallet changes and broadcasts sanitized results.

Poker uses server-side table authority. Actions flow through `server/src/poker/services/pokerActionOrchestrator.service.ts` and the gateway; table logic lives under `server/src/logic/`. Hidden bets are isolated under `server/src/poker/hiddenBets/`, with separate pricing, placement, settlement and ledger logic tied to poker street transitions.

Blackjack, roulette and slots are separated from the poker engine. Blackjack has solo and multiplayer flows under `server/src/blackjack/`; roulette and slot logic use dedicated modules and API routes. Outcomes, stake validation, payout calculation and wallet updates remain server-side.

Tournaments separate creation, registration, waiting state, table assignment, progression, elimination, finalization and rewards. Client screens live under `client/src/features/tournament/`, while orchestration lives under `server/src/tournament/`. Winner side bets are handled by dedicated tournament services so settlement follows the real tournament result.

Social features combine durable REST operations and live Socket.IO updates: friends, invitations, remove friend, block/unblock, reports, chat access, activity status and blocked-user warnings. Progression features such as XP, badges, daily challenges, login streaks, leaderboards, gift codes and recharge flows are separate modules with server-side eligibility and balance validation.

Monetary integrity follows one rule: client balances are display data, while the server and

database are the source of truth. Chip-changing operations use server-side settlement, Prisma persistence, ledger-oriented records, idempotency where needed and domain-specific reward or payout services.

S External references and compendium

- Initial formal requirements PDF: `Docs/Cahier_de_charge_6A_PI.pdf` (not embedded here).
- Auto-generated wide-coverage technical compendium: `Docs/Rapport_final/RAPPORT_TECHNIQUE_COMPLET.md` (use as index; verify claims against code).
- Security dossiers: `Docs/Architecture/SECURITE_A_TO_Z.md`, `Docs/security/`.
- HTTP and socket maps: `Docs/detail_logique/network/01_http_api_map.md`, `Docs/detail_logique/network/02_socket_events_map.md`.
- Combination analysis report: `Docs/final_report/QUANTUM_COMBI.adoc`.
- Slot RTP and outcome algebra (AsciiDoc): `Docs/SLOT_MACHINE.adoc`.
- Roulette payouts and expectation: `Docs/final_report/ROULETTE.adoc`.

References

[1] Meta. *React documentation* (React 19). https://react.dev/ (accessed May 2026).

[2] Microsoft. *TypeScript handbook.* https://www.typescriptlang.org/docs/ (accessed May 2026).

[3] Vite contributors. *Vite guide.* https://vite.dev/guide/ (accessed May 2026).

[4] OpenJS Foundation. *Node.js runtime documentation.* https://nodejs.org/docs/latest/api/ (accessed May 2026).

[5] OpenJS Foundation. *Express web framework.* https://expressjs.com/ (accessed May 2026).

[6] Socket.IO project. *Socket.IO server and client (v4).* https://socket.io/docs/v4/ (accessed May 2026).

[7] Prisma Data Inc. *Prisma ORM documentation.* https://www.prisma.io/docs (accessed May 2026).

[8] PostgreSQL Global Development Group. *PostgreSQL documentation.* https://www.postgresql.org/docs/ (accessed May 2026).

[9] Redis Ltd. *Redis documentation.* https://redis.io/docs/ (accessed May 2026).

[10] Redux team. *Redux Toolkit and RTK Query.* https://redux-toolkit.js.org/ (accessed May 2026).

[11] i18next authors. *i18next documentation.* https://www.i18next.com/ (accessed May 2026).

[12] Zod maintainers. *Zod schema validation.* https://zod.dev/ (accessed May 2026).

[13] OpenJS Foundation. *Jest testing framework.* https://jestjs.io/ (accessed May 2026).

[14] Vitest contributors. *Vitest.* https://vitest.dev/ (accessed May 2026).

[15] Microsoft. *Playwright end-to-end testing.* https://playwright.dev/ (accessed May 2026).

[16] Tailwind Labs. *Tailwind CSS documentation.* https://tailwindcss.com/docs (accessed May 2026).

[17] Schwaber, K., Sutherland, J. *The Scrum Guide* (2020). https://scrumguides.org/scrum-guide.html (accessed May 2026).

[18] GitLab Inc. *GitLab CI/CD documentation.* https://docs.gitlab.com/ee/ci/ (accessed May 2026).

[19] Université de Strasbourg. *Faculté des mathématiques et informatique.* https://mathinfo.unistra.fr/ (accessed May 2026).